

**MASARYKOVA
UNIVERZITA**

FAKULTA INFORMATIKY

**Automatické aktualizace
integračních testů webových
aplikací pomocí LLM**

Diplomová práce

BC.LUKÁŠ HAVLÍČEK

Brno, podzim 2025

**MASARYKOVA
UNIVERZITA**

FAKULTA INFORMATIKY

**Automatické aktualizace
integračních testů webových
aplikací pomocí LLM**

Diplomová práce

BC.LUKÁŠ HAVLÍČEK

Vedoucí práce: Mgr. Marek Grác, Ph.D.

Katedra strojového učení a zpracování dat

Brno, podzim 2025



Prohlášení

Prohlašuji, že tato práce je mým původním autorským dílem, které jsem vypracoval samostatně. Všechny zdroje, prameny a literaturu, které jsem při vypracování používal nebo z nich čerpal, v práci řádně cituji s uvedením úplného odkazu na příslušný zdroj.

Při přípravě této práce jsem využil nástroje umělé inteligence ChatGPT a Google Gemini pro konzultace a vylepšení stylu textu, a model Codex pro pomoc při programování. Po použití těchto nástrojů jsem provedl kontrolu obsahu a přebírám za něj plnou zodpovědnost.

Bc.Lukáš Havlíček

Vedoucí práce: Mgr. Marek Grác, Ph.D.

Poděkování

Poděkování vedoucímu práce Mgr. Marku Grácovi, Ph.D., za vedení a podněty, kolegům za diskuse a rodině za podporu.

Shrnutí

Drobná změna uživatelského rozhraní často způsobí selhání integračních testů a kroků při scrapingu. Cílem práce je navrhnout a ověřit LLM agenta, který dokáže tato selhání diagnostikovat a automaticky navrhnout minimální, auditovatelnou opravu. Navržené řešení vychází z architektury planner–executor–critic a kombinuje jazykové modely s deterministickým prováděním testů.

Klíčová slova

LLM, agent, integrační testy, Playwright, Selenium, self-healing testy, scraping, DOM, ARIA, determinismus, CI/CD

Obsah

1	Úvod	1
2	Textová východiska	2
2.1	Testování webových aplikací	3
2.1.1	Selenium	4
2.1.2	Playwright	5
2.1.3	Cypress	5
2.2	Problém údržby testů	6
2.2.1	Příklady flakiness a detekce rozbitých testů . . .	7
2.3	LLM v kontextu testování a automatizace	8
2.3.1	Agenti pro interakci s uživatelským rozhraním .	8
2.3.2	Kombinace deterministických kroků a LLM rea- soning	9
2.3.3	Limity a výzvy použití LLM	10
2.3.4	Hallucinace u jazykových modelů	11
2.3.5	Principy správného promptování	11
3	Návrh systému pro automatické opravy testů	13
3.1	Architektura systému	13
3.2	Deterministické jádro	14
3.3	LLM komponenta	15
3.4	Orchestrátor	17
3.5	Datový tok	18
3.6	Reprezentace znalostí	18
3.7	Omezení návrhu a Shadow DOM	19
4	Implementace proof-of-concept	21
4.1	Přehled implementace	21
4.2	Popis modulů	22
4.2.1	orchestrator.ts	22
4.2.2	adapter.ts	22
4.2.3	scraping-orsr.ts	23
4.2.4	prompts.ts	23
4.2.5	specs/*.json	24
4.3	Popis pipeline	25

4.4	Formát promptů a strategie řízení LLM	26
4.5	Integrace s Playwrightem	27
4.6	Postup adaptace existujícího scraperu	28
4.7	Ukázky běhu a diagnostiky	32
5	Experimentální vyhodnocení	35
5.1	Testovací datová sada	35
5.2	Metodika měření	35
5.3	Diskuse výsledků	36
5.3.1	Souhrnné kvantitativní výsledky	36
5.3.2	Analýza jednotlivých scénářů	37
5.4	Limity a rizika	39
6	Ověření postupu adaptace na systému RPVS	40
6.1	Charakteristika cílového systému	40
6.2	Aplikace metodiky adaptace	40
6.2.1	Krok 1: Centralizace lokátorů	41
6.2.2	Krok 2 a 3: Refaktoring a obohacení chyb	41
6.2.3	Krok 4 a 5: Integrace hybridní vrstvy	41
6.3	Zjištění a optimalizace během adaptace	41
6.4	Výsledky ověření	42
6.5	Závěr k adaptaci	42
7	Budoucí práce	44
8	Závěr	46
	Bibliografie	47
A	Přílohy	50
A.1	Šablony systémových promptů	50
A.2	Časová náročnost	57
A.3	Schéma architektury systému	57
A.4	Obsah elektronické přílohy	59

Seznam tabulek

5.1	Souhrnné výsledky experimentálních běhů (N=400)	36
6.1	Výsledky validace adaptovaného RPVS scraperu (N=400)	42
A.1	Srovnání časové náročnosti opravy chyby (ORSR)	57

Seznam obrázků

A.1 Schéma datového toku a kontrolních mechanismů v systému 58

1 Úvod

Automatizované testování webových aplikací je nedílnou součástí moderního vývojového procesu. Jeho cílem je opakovaně ověřovat správnou funkčnost aplikace pomocí předem definovaných scénářů, které simulují chování skutečného uživatele. V praxi se k tomuto účelu využívají nástroje jako Selenium nebo Playwright, jež umožňují ovládání prohlížeče prostřednictvím skriptů. Takový přístup zajišťuje deterministické a spolehlivé testování, avšak i drobná úprava uživatelského rozhraní, například změna názvu třídy nebo přeuspořádání prvků DOM, může způsobit selhání testů. Údržba testovací sady se tak často stává časově náročnou a repetitivní činností.

V posledních letech se do oblasti testování začaly prosazovat velké jazykové modely (LLM), které dokážou analyzovat textový kontext, interpretovat chybová hlášení a navrhnout vhodné úpravy kódu. Tyto schopnosti otevírají možnost využít model nejen k vytváření testů, ale také k jejich opravám. Při správném zapojení může model pomoci s automatickou detekcí selhání a návrhem opravy bez nutnosti zásahu vývojáře. Výzkumné projekty jako GPT-Engineer nebo SWE-Agent ukazují, že LLM lze integrovat do prostředí vývoje softwaru jako agenty schopné analyzovat stav systému a generovat odpovídající kroky.

Cílem této práce je navrhnout a implementovat systém, který dokáže na základě existujících testů automaticky identifikovat chyby vzniklé v důsledku změn uživatelského rozhraní a pomocí jazykového modelu navrhnout jejich opravu. Výsledkem by měl být nástroj, který kombinuje deterministické provádění testů s adaptivním vyhodnocováním pomocí LLM, čímž se zrychlí proces údržby a sníží množství manuálních zásahů při drobných změnách aplikace.

Navržené řešení vychází z architektury *planner-executor-critic*, kde plánovací část využívá jazykový model k návrhu úpravy testu, deterministická část provádí a vyhodnocuje změnu a výsledky jsou následně validovány. Implementace je zpracována jako proof-of-concept rozšíření existujícího systému pro scraping a integrační testování webových aplikací.

2 Textová východiska

S rostoucí složitostí moderních webových aplikací se výrazně zvyšují i nároky na zajištění jejich spolehlivosti. Při častém nasazování nových verzí je nutné po každé změně kódu automaticky ověřit, že aplikace nadále funguje podle očekávání. Tuto úlohu plní automatizované testy, které simulují chování uživatele a kontrolují, zda aplikace reaguje očekávaným způsobem.

V praxi se však ukazuje, že i při vysoké míře automatizace zůstává údržba testů časově náročná a náchylná k chybám. I drobná úprava vzhledu nebo struktury uživatelského rozhraní, například změna názvu třídy, přeuspořádání prvků v DOM nebo modifikace identifikátoru, může způsobit, že test přestane fungovat. Přitom samotná funkčnost aplikace zůstává nezměněna. Taková situace vede k selhání testů, které neodhalují skutečnou chybu, ale pouze ztrátu vazby mezi testem a testovaným prvkem. Tyto případy se označují jako *false negatives* a představují jeden z hlavních problémů automatizovaného testování webových aplikací(1, 2).

Tradiční přístupy k testování, jako je používání nástrojů Selenium, Playwright nebo Cypress, jsou založeny na deterministickém provádění přesně definovaných kroků.(3, 4, 5) Tyto nástroje umožňují skriptovat akce uživatele, jako je kliknutí, vyplnění formuláře nebo ověření textu, a spolehlivě kontrolovat reakce aplikace. Deterministická povaha těchto nástrojů zajišťuje reprodukovatelnost výsledků, což je z pohledu kvality testování nezbytné. Jejich slabinou je však právě nedostatek adaptability, testovací skript dokáže pouze zopakovat předem daný postup a neumí se přizpůsobit drobným změnám v rozhraní.

Při větších projektech může mít takové chování výrazné důsledky. Každá změna v aplikaci může vyvolat řetězec neúspěšných testů, které je nutné ručně analyzovat a upravit. Tento proces často představuje více než polovinu času věnovaného údržbě testovací sady a snižuje celkovou efektivitu vývojového cyklu.

V reakci na tento problém se začínají objevovat nové přístupy, které do procesu testování zapojují metody strojového učení a jazykové modely(6, 7, 8). Ty umožňují analyzovat kontext selhání, identifikovat příčinu problému a navrhnout automatickou opravu testu. Cílem není

nahradit deterministické testování, ale rozšířit ho o adaptivní vrstvu, která dokáže zachytit změny v rozhraní a přizpůsobit testovací scénáře bez nutnosti zásahu člověka.

Následující část se proto zaměřuje na popis tradičních testovacích frameworků a jejich principů, které tvoří základ současných metod automatizace testů webových aplikací. Následně jsou představeny přístupy využívající velké jazykové modely (LLM) a agentní architektury, které umožňují adaptivní chování testů a představují směr, jímž se tato oblast v současnosti vyvíjí.

2.1 Testování webových aplikací

Testování webových aplikací je nedílnou součástí vývoje a údržby softwaru. Jeho cílem je ověřit, že jednotlivé části systému i jejich vzájemná spolupráce fungují podle očekávání a že aplikace zůstává stabilní i při změnách prostředí nebo vstupů. U webových systémů se testování potýká se specifiky, která u běžných aplikací obvykle nevznikají – například s asynchronním načítáním dat, různým chováním prohlížečů nebo závislostí na externích službách. Automatizace je proto klíčová: umožňuje opakovatelné a reprodukovatelné testy, které pomáhají zachytit chyby v rané fázi vývoje a zkracují dobu zpětné vazby (1, 2).

Testy lze rozdělit podle úrovně, na které ověřují chování systému. Nejnižší úroveň představují *unit testy*, které se zaměřují na jednotlivé funkce nebo třídy v izolaci. Ověřují správnost logiky bez závislosti na dalších částech systému a poskytují rychlou zpětnou vazbu při vývoji.

Nad nimi následují *integrační testy*, které ověřují, že spolu jednotlivé komponenty správně komunikují. Sledují tok dat mezi moduly, API nebo databázemi a pomáhají odhalit chyby na rozhraních, které by v izolaci zůstaly skryté. Tato úroveň testů bývá pomalejší, ale lépe odráží skutečné fungování systému v běhu (9).

End-to-end testy (E2E) představují nejvyšší úroveň a simulují skutečné chování uživatele. V případě webových aplikací probíhají přímo v prohlížeči, kde testovací nástroj ovládá rozhraní podobně jako člověk – kliká, vyplňuje formuláře a kontroluje reakce systému. Tím se ověřuje celá cesta dat napříč aplikací. Tyto testy poskytují nejvyšší jistotu správné funkčnosti, ale současně jsou časově náročnější a citlivější na

změny uživatelského rozhraní, například na přejmenování tlačítek nebo úpravu struktury DOM. Tento jev se označuje jako *flakiness*, tedy nestabilita testů bez zjevné změny kódu (10, 2).

Jednotlivé úrovně testů se obvykle propojují do tzv. *testovací pyramidy*, která popisuje ideální rozložení poměru mezi nimi. Základem jsou rychlé a četné unit testy, nad nimi stojí menší počet integračních testů a vrchol tvoří omezená sada end-to-end testů. Takto navržená hierarchie zajišťuje rovnováhu mezi rychlostí, pokrytím a spolehlivostí testovacího procesu (9).

V oblasti webových aplikací se v posledních letech prosazují nástroje, které se zaměřují na stabilitu a automatizaci testů. Frameworky jako Playwright, Selenium nebo Cypress umožňují spouštět testy přímo v prohlížeči, automaticky čekat na načtení prvků a používat sémanticky robustní selektory. Díky tomu jsou testy méně náchylné k chybám způsobeným drobnými změnami v rozhraní a jejich údržba je efektivnější (4, 3, 5).

2.1.1 Selenium

Selenium je jeden z nejdéle používaných nástrojů pro automatizaci webových prohlížečů a testování uživatelského rozhraní. Jeho jádrem je komponenta *WebDriver*, která umožňuje ovládat prohlížeč nativně – tedy stejným způsobem, jakým s ním interaguje uživatel. Tento přístup zajistil Seleniiu širokou podporu napříč jazyky a platformami a stal se de facto standardem v oblasti end-to-end testování webových aplikací (3).

Pro zajištění stability testů Selenium využívá mechanismy *implicitních* a *explicitních* čekání. Implicitní čekání definuje globální časový limit, během něhož se WebDriver opakovaně pokouší najít prvek, zatímco explicitní čekání (*Expected Conditions*) umožňuje cíleně vyčkat na konkrétní stav, například viditelnost nebo klikatelnost prvku (11, 12). Dokumentace doporučuje tyto přístupy nekombinovat, aby nedocházelo k nečekaným prodlevám a konfliktům v načasování.

Výhodou Selenium je jeho otevřená architektura a kompatibilita s různými prohlížeči i jazykovými bindingy (Python, Java, JavaScript aj.), což z něj činí univerzální nástroj vhodný pro rozsáhlé testovací infrastruktury. Slabinou však zůstává vyšší náchylnost k chybám synchronizace a křehkost selektorů, zejména pokud jsou konstruovány na

základě hlubokých cest v DOM nebo pořadových indexů. Tyto faktory vedly k vývoji novějších frameworků, jako je Playwright, které kladou větší důraz na robustní lokátory a automatickou synchronizaci běhu testů.

2.1.2 Playwright

Playwright představuje moderní open-source nástroj pro automatizaci webových prohlížečů vyvíjený společností Microsoft. Na rozdíl od starších frameworků, jako je Selenium WebDriver, poskytuje jednotné rozhraní pro všechny hlavní prohlížeče (Chromium, Firefox a WebKit) a klade důraz na determinismus a stabilitu testů. Základním principem Playwrightu je koncept *auto-waitingu*, který eliminuje potřebu explicitního čekání na načtení nebo zpřístupnění prvků. Před každou akcí je automaticky vyhodnocena tzv. *actionability* – tedy zda je prvek viditelný, povolený a připravený k interakci. Tím se minimalizuje riziko nestabilních běhů způsobených asynchronním načítáním stránky (4).

Klíčovou součástí API jsou tzv. *locators*, které nahrazují klasické selektory a umožňují opakované a konzistentní vyhledávání prvků v DOM. Lokátory mohou být založeny na sémantických vlastnostech, například pomocí metod `getByRole` či `getByLabel`, které využívají principy přístupnosti (WAI-ARIA). Tento přístup zvyšuje odolnost testů vůči kosmetickým změnám rozhraní a podporuje jejich dlouhodobou údržbu (13). Playwright rovněž nabízí vestavěný *Trace Viewer*, nástroj pro detailní analýzu běhů testů, který umožňuje zpětně zobrazit všechny provedené akce, síťovou komunikaci i snímky obrazovky jednotlivých kroků. To významně usnadňuje diagnostiku selhání a přispívá k auditovatelnosti testovacího procesu (14).

2.1.3 Cypress

Cypress je framework pro end-to-end testování, který spouští testy přímo v prohlížeči společně s testovanou aplikací. Díky tomu poskytuje detailní kontrolu nad interakcemi a současně zabudovanou strategii čekání, příkazy i aserce se automaticky opakují a blokují průběh, dokud aplikace nedosáhne očekávaného stavu. Timeouty lze nastavovat globálně i per-příkaz, takže není nutné vkládat pevné

prodlevy, dokumentace výslovně doporučuje vyhýbat se zbytečným `wait()` a spoléhat na implicitní retry chování. To v praxi snižuje flakiness způsobený asynchronním vykreslováním a síťovými latencemi (15, 5).

Významnou součástí je práce se sítí. Pomocí `cy.intercept()` lze požadavky sledovat i stubovat, včetně úpravy odpovědi (status, tělo, hlavičky), což usnadňuje deterministické scénáře bez závislosti na externích API. Zároveň je vhodné mít na paměti limity architektury — Cypress nativně neřídí více oken současně a standardně vykonává příkazy v rámci jednoho originu, pro multi-origin scénáře existuje `cy.origin()` jako řízená výjimka (16, 17).

2.2 Problém údržby testů

S rostoucí velikostí a dynamikou webových aplikací se zvyšuje i objem testů, které je nutné průběžně udržovat. Automatizované integrační a end-to-end testy bývají citlivé na i drobné změny v uživatelském rozhraní, které nemají vliv na samotnou funkcionalitu aplikace. Přesun prvku v rámci DOM, úprava textu, přejmenování třídy nebo přidání nového wrapperu mohou způsobit selhání testu, i když aplikace jako celek funguje správně. V praxi pak vývojáři tráví značnou část času opravami testů, které se „rozbily“ po vizuální nebo strukturální úpravě stránky. Tento typ údržby bývá repetitivní a představuje dlouhodobý problém při škálování testovacích sad (2).

Jedním z nejčastějších jevů při automatizovaném testování je *flakiness* — nestabilita testů, které při opakovaném spuštění střídavě procházejí a selhávají bez změny kódu. Příčinou může být pomalé načítání obsahu, závod mezi asynchronními požadavky, krátká prodleva při vykreslování nebo vliv animací. Takové testy snižují důvěryhodnost výsledků, prodlužují vývojové cykly a komplikují analýzu chyb v CI pipeline. Z pohledu praxe bývá složité rozlišit, zda se jedná o náhodné selhání, nebo skutečný problém v kódu, což vede k opakovanému spouštění testů a zbytečným nákladům na běh infrastruktury (1, 10).

Moderní frameworky, jako Playwright nebo Cypress, se snaží tyto problémy omezit automatickou synchronizací příkazů s reálným stavem uživatelského rozhraní. Testovací příkaz se spustí až tehdy, když

je prvek „akceschopný“ tedy viditelný, dostupný a neblokováný jiným prvkem. Tím se minimalizuje potřeba ručního nastavování prodlev nebo opakování akcí a testy se stávají stabilnějšími. Kromě toho nástroje ukládají artefakty z běhu testu (logy, screenshoty, trace), které usnadňují zpětnou analýzu a lokalizaci problému (4, 5).

Vedle náhodné nestability se v praxi vyskytují i trvale nefunkční testy, které po změně uživatelského rozhraní selhávají konzistentně při každém běhu. Takové případy se dají odhalit analýzou trendů v CI nástrojích nebo sledováním dlouhodobě neúspěšných testů. I když většinu těchto chyb lze opravit úpravou selektorů nebo aktualizací očekávaných textů, při rozsáhlých projektech se jedná o časově náročný proces, který vyžaduje manuální zásah. Tento problém vede k hledání automatizovaných přístupů, které by dokázaly rozpoznat příčinu selhání a navrhnout vhodnou opravu testu bez zásahu člověka. Výzkum v této oblasti se zaměřuje na využití strojového učení a velkých jazykových modelů (LLM), které mohou na základě artefaktů z běhu testu identifikovat rozbité části kódu a generovat jejich opravenou verzi (7).

2.2.1 Příklady flakiness a detekce rozbitých testů

Nestabilní testy se v praxi projevují několika typickými způsoby. Nejčastějším případem je situace, kdy test selže z důvodu zpožděného načítání prvku nebo jeho dočasné nedostupnosti. Typicky se jedná o tlačítka, která se zobrazují až po vykreslení komponenty, nebo o dynamicky načítaný obsah po síťovém požadavku. Při příliš brzkém volání akce, například `click()`, test skončí chybou, přestože by při manuálním kliknutí fungoval správně. Podobně mohou problémy způsobovat animace, přechody či překrývající se prvky, které dočasně brání interakci (2).

Další častou příčinou flakiness je nedeterministické chování samotného testovacího prostředí. Pokud test závisí na externím API, náhodném pořadí dat nebo časovém limitu, může selhávat nepravidelně i bez úprav kódu. Tato selhání bývají nejtěžší na odhalení, protože se neprojevují konzistentně – jeden běh testu uspěje, další selže, a výsledek se často nedá jednoduše reprodukovat. Detekce takových testů se obvykle opírá o statistiku neúspěšných běhů, sledování

trendů v CI/CD pipeline nebo opakované spouštění testů s fixním seedem (1).

Rozbité testy, tedy ty, které trvale selhávají po změně uživatelského rozhraní, se dají často identifikovat už při prvním spuštění po nasažení nové verze. V nástrojích jako Playwright nebo Cypress bývá k dispozici detailní výpis kroků, screenshoty a trace, které umožňují rychle určit, který krok selhal a proč. Typickým indikátorem bývá selhání při hledání prvku (`locator not found`) nebo chyba interakce s neaktivním tlačítkem (`element not actionable`). Kombinace těchto informací s historií předchozích běhů umožňuje vývojářům odlišit nahodilé chyby od trvale rozbitých testů a zacílit údržbu jen na skutečně problémové části (14).

2.3 LLM v kontextu testování a automatizace

V posledních letech se velké jazykové modely (LLM) začaly uplatňovat také v oblasti automatizace softwarového testování. Tradiční testovací nástroje se opírají o deterministické postupy s pevně danými kroky, zatímco LLM jsou schopny pracovat s kontextem a formulovaným popisem problému. Díky tomu dokážou nejen vykonávat testovací scénáře, ale také analyzovat chyby a navrhnout vhodné opravy. Tento přístup přináší vyšší míru autonomie a umožňuje zefektivnit proces údržby testů.

2.3.1 Agenti pro interakci s uživatelským rozhraním

LLM se často využívají v kombinaci s nástroji, které modelu umožňují interakci s uživatelským rozhraním nebo vývojovým prostředím. Tyto systémy bývají označovány jako *UI agenti* a fungují na principu cyklu pozorování, rozhodnutí a akce. Agent sleduje aktuální stav prostředí, například webového prohlížeče, interpretuje textové i vizuální prvky a podle cíle generuje další kroky.

Mezi praktické příklady systémů, které využívají LLM k řízení akcí nad uživatelským rozhraním nebo programovým prostředím, patří projekty *GPT-Engineer*, *SWE-Agent* a *Voyager*. Každý z nich ukazuje jiný přístup k integraci jazykového modelu s deterministickým prostředím.

Projekt *GPT-Engineer* (18) je otevřený nástroj určený pro generování a úpravu zdrojového kódu na základě textového zadání. Uživatel definuje popis požadované aplikace, na jehož základě model analyzuje strukturu projektu a generuje potřebné soubory. Výhodou tohoto přístupu je možnost iterativního vylepšování výstupu – model si uchovává kontext a na základě zpětné vazby upravuje výsledný kód. Tento koncept se uplatňuje i při automatickém vytváření testovacích scénářů nebo refaktoringu existujících projektů.

SWE-Agent (19) představuje specializovaného agenta vyvinutého týmem Princeton NLP pro automatizované řešení softwarových úloh. Agent využívá LLM k analýze zadání, procházení zdrojového kódu a provádění oprav v reálném repozitáři. Každý krok je prováděn v kontrolovaném prostředí, kde model na základě výsledku testů rozhoduje o dalším postupu. Tento princip připomíná učení s posilováním, kdy agent postupně optimalizuje svůj výkon podle dosaženého výsledku.

Projekt *Voyager* (20) rozšiřuje využití LLM i do prostředí simulovaných světů. Agent je schopen samostatně objevovat herní prostředí (např. Minecraft), plánovat sekvenci akcí a ukládat naučené dovednosti pro další použití. Architektura systému je založena na smyčce pozorování, plánování a akce, což z něj činí příklad obecného agenta schopného adaptace. Koncepce použitá ve Voyageru se uplatňuje i v testování uživatelských rozhraní, kde může model z podobných principů vyvozovat potřebné interakce s prvky stránky.

Společným prvkem uvedených projektů je propojení jazykového modelu s prostředím, které provádí reálné akce. Tímto způsobem lze model využít nejen pro generování textu, ale i pro autonomní provádění úloh vyžadujících rozhodování a zpětnou vazbu.

2.3.2 Kombinace deterministických kroků a LLM reasoning

Pro testování webových aplikací se jako nejúčinnější ukazuje hybridní přístup, který kombinuje deterministické provádění kroků s jazykovým uvažováním modelu. Deterministická část zajišťuje spolehlivé provedení přesně definovaných akcí, jako je kliknutí na prvek, vyplnění formuláře nebo kontrola textu na stránce. LLM následně doplňuje schopnost interpretovat kontext, analyzovat chybová hlášení a navrhnout možné opravy testů. Tím se eliminuje nutnost manuálního zásahu

v případech, kdy test selže kvůli drobným změnám v uživatelském rozhraní.

V praxi se tento přístup často realizuje pomocí architektury *planner-executor-critic*, která vychází z principů popsaných u agentů ReAct a Reflexion (7, 8). Plánovací část využívá jazykový model k určení sekvence kroků potřebných k dosažení cíle, vykonavatel (executor) tyto kroky provádí prostřednictvím testovacího nástroje, například Playwright nebo Selenium, a hodnoticí komponenta (critic) vyhodnocuje úspěšnost provedení. Pokud výsledek neodpovídá očekávanému stavu, model získává nové informace z logů, DOM struktury nebo screenshotů a navrhuje opravu testu.

Hybridní řešení umožňuje spojit stabilitu deterministických metod s adaptivitou jazykového modelu (20). Při opakovaných bězích testů zůstává zachována reprodukovatelnost výsledků, protože deterministická část je neměnná, zatímco LLM může flexibilně reagovat na neočekávané situace, jako jsou drobné úpravy rozhraní nebo zpoždění načítání prvků. Tento přístup zkracuje dobu potřebnou pro údržbu testů a snižuje množství selhání způsobených neaktuálními selektory.

Dalším využitím hybridního principu je tzv. *self-healing testing* (21), kdy systém po detekci chyby automaticky vyhledá alternativní prvek v rozhraní, který odpovídá původnímu selektoru, a pokračuje v testu. Taková oprava může být následně potvrzena vývojářem, což zajišťuje kontrolu nad kvalitou automatických úprav. LLM se zde uplatňuje zejména při rozpoznávání záměru testu a při odhadu nejvhodnější opravy, zatímco deterministická část dohlíží na konzistenci a bezpečné provedení.

Spojení těchto dvou přístupů tak přináší rovnováhu mezi spolehlivostí a flexibilitou. Deterministické metody poskytují jistotu reprodukovatelnosti a dohledatelnosti výsledků, zatímco LLM umožňuje rychlou adaptaci na změny a rozšiřuje možnosti automatizace testování nad rámec klasických skriptovacích přístupů.

2.3.3 Limity a výzvy použití LLM

Použití LLM v testovací praxi naráží na několik omezení. Jedním z hlavních problémů je tzv. *hallucination*, kdy model generuje chybné nebo smyšlené výstupy. V oblasti testování to může znamenat návrh neplatného selektoru či chybné úpravy testovacího kódu. Dalším pro-

blémem je *nedeterminismus*, kdy stejný vstup nemusí vždy vést ke stejnému výstupu, což komplikuje auditovatelnost výsledků. LLM navíc vyžadují výpočetně náročný provoz a lidskou kontrolu výstupů, což omezuje jejich plnou automatizaci. Přesto se ukazuje, že vhodná kombinace jazykového modelu s deterministickým systémem dokáže zkrátit dobu údržby testů a zvýšit jejich odolnost vůči změnám rozhraní (6).

2.3.4 Hallucinace u jazykových modelů

Jedním z přetrvávajících problémů jazykových modelů je tvorba tzv. *hallucinací*, tedy tvrzení, která působí věrohodně, ale nejsou pravdivá. Podle výzkumu OpenAI (22) dochází k halucinacím především proto, že modely jsou během trénování i vyhodnocování odměňovány za správné odpovědi, nikoli za přiznání nejistoty. Tím se vytváří prostředí, ve kterém model „hádá“, i když si není jistý výsledkem, místo aby odpověď odmítl nebo požádal o upřesnění.

Článek dále uvádí, že přesnost modelu sama o sobě nestačí ke snížení halucinací, protože vysoká přesnost může být dosažena i díky náhodným odhadům. Při hodnocení výkonu se proto doporučuje zohlednit také schopnost modelu správně vyjádřit nejistotu. OpenAI navrhuje, aby chybným, sebevědomým odpovědím byla přiřazena vyšší penalizace než případům, kdy model odmítne odpovědět.

Z pohledu praktického využití znamená tento poznatek nutnost upravit způsob, jakým jsou modely hodnoceny a integrovány do testovacích systémů. Pokud by testovací agenti založení na LLM dokázali rozpoznat situace, kdy je pravděpodobnost chyby vysoká, a místo odhadu zvolili „abstenci“, snížila by se pravděpodobnost zavádějících výstupů. Tento přístup podporuje přesnější a spolehlivější fungování modelů v prostředí, kde jsou důležité ověřitelné výsledky a možnost zpětné kontroly.

2.3.5 Principy správného promptování

Kromě samotné architektury modelu má na kvalitu výstupů výrazný vliv i způsob, jakým je model dotazován. Dokumentace OpenAI (23) zdůrazňuje, že správně formulovaný vstup (*prompt*) je klíčový pro dosažení konzistentních a přesných výsledků. Prompt engineering

představuje proces tvorby zadání tak, aby model jednoznačně porozuměl cíli, kontextu a požadovanému formátu odpovědi.

Podle doporučení OpenAI je vhodné kombinovat několik osvědčených postupů: jasné vymezení rolí zpráv, oddělení pokynů pro model (*developer messages*) od uživatelských vstupů (*user messages*) a používání strukturovaného formátu se sekcemi jako identita, instrukce, příklady a kontext. Tyto prvky pomáhají modelu lépe pochopit záměr uživatele a omezují nejednoznačnost, která často vede k chybám nebo halucinacím.

Dále se doporučuje využívat verzování promptů a testování jejich výkonu pomocí metrik (*evals*), aby bylo možné sledovat, jak se mění přesnost a stabilita odpovědí při úpravách zadání. U složitějších úloh může být přínosné doplnit prompt o několik vzorových vstupů a výstupů (*few-shot learning*), z nichž model odvodí očekávaný vzorec chování.

Z hlediska spolehlivosti je důležité volit modely s fixovanou verzí a přesně specifikovat požadavky. GPT modely, například GPT-5, reagují nejlépe na explicitní pokyny, zatímco tzv. *reasoning* modely zvládají úlohy s obecnějšími instrukcemi. OpenAI rovněž umožňuje využít parametr `optimize=true`, který interně upravuje způsob zpracování promptu tak, aby model efektivněji sledoval zadané instrukce a poskytoval konzistentnější výsledky (**openai-optimize-param**). Tato možnost je vhodná zejména pro produkční aplikace, kde je žádoucí stabilita výstupu napříč jednotlivými generacemi modelu.

Přesné a strukturované promptování spolu s vhodnou konfigurací modelu tak představuje jeden z nejefektivnějších způsobů, jak omezit výskyt halucinací a zlepšit interpretovatelnost i opakovatelnost výsledků.

3 Návrh systému pro automatické opravy testů

Údržba automatizovaných integračních testů a kroků při scrapingu webových aplikací je často znesnadněna přímou závislostí na konkrétní struktuře DOM (2). I drobné úpravy uživatelského rozhraní mohou vést k selhání testů, přestože samotná funkcionality aplikace zůstává nezměněna. Cílem návrhu popsaného v této kapitole je systém schopný tato selhání nejen detekovat, ale také automaticky navrhnout a ověřit jejich opravu s využitím velkých jazykových modelů (LLM). Systém staví na existujícím deterministickém řešení pro scraping dat, které rozšiřuje o adaptivní fallback mechanismus.

3.1 Architektura systému

Navržený systém kombinuje spolehlivost deterministického provádění s flexibilitou jazykových modelů. Jeho architektura vychází z hybridního přístupu, který zachovává původní, rychlé deterministické jádro, ale doplňuje ho o inteligentní záložní vrstvu. Skládá se ze tří hlavních komponent:

První komponentou je **Deterministické jádro**, které zodpovídá za provádění původních, předem definovaných kroků testu nebo scrapingu pomocí nástroje Playwright (13). Zahrnuje sadu funkcí pro interakci se stránkou (navigaci, extrakci dat) a repozitář lokátorů.

Druhou komponentou je **LLM komponenta**, která se aktivuje v případě selhání deterministického jádra. Jejím úkolem je analyzovat kontext chyby (chybové hlášení, stav DOM) a pomocí LLM navrhnout opravu, buď jako upravený plán kroků (pro navigaci), nebo jako přímo extrahovaná data (pro extrakci). Zahrnuje adaptér pro komunikaci s LLM API, systém promptů definující strategii oprav a specifikace cílů pro jednotlivé úlohy.

Poslední komponentou je **Orchestrátor**, který řídí celkový tok. Rozhoduje o použití deterministické nebo fallbackové (LLM) cesty, spouští testy/extrakci, vyhodnocuje výsledky a provádí následné kontroly (validace dat, kontrola konzistence s historií).

Tento návrh umožňuje zachovat výhody deterministického testování (rychlost, reprodukovatelnost) pro případy, kdy se rozhraní

nemění, a zároveň poskytuje robustní záložní mechanismus pro situace, kdy dojde k drobným úpravám HTML struktury.

3.2 Deterministické jádro

Základem systému je sada TypeScript funkcí implementovaných v souboru `scraping-orsr.ts`, které využívají knihovnu Playwright k automatizaci interakce s webovým portálem Obchodného registra Slovenskej republiky (ORSR). Tyto funkce zajišťují základní operace, které lze rozdělit na navigační a extrakční.

Pro interakci se stránkou jsou využívány **lokátory** (v této práci označované také jako *selektory*). Jedná se o řetězce, které jednoznačně identifikují prvky na stránce, například

`'role=link[name=/identifikačního čísla/i]'`. V rámci navržené architektury plní tyto lokátory dvojí roli. Zaprvé, jsou přímo využívány deterministickými funkcemi (např. `findListByName`) pro přímou interakci se stránkou. Zadruhé, slouží jako základní stavební kameny pro sestavování deterministických plánů (např. ve funkci `buildDeterministicCompanyNamePlan`), které jsou následně v případě selhání předány LLM komponentě k opravě. Právě toto dvojí využití činí jejich centralizaci klíčovou. Zásadním krokem návrhu proto bylo sjednocení lokátorů do souboru `src/orsr-selectors.ts`. Tím je zajištěno, že jak deterministická logika, tak základní plán pro LLM orchestrátor sdílejí stejnou definici, což zásadně usnadňuje údržbu při změnách cílové webové stránky.

Funkce pro **navigaci**, jako `findListByName`, `findDetailByICO` či `findDetailByCompanyName`, implementují pevně dané sekvence kroků (kliknutí na odkazy, vyplnění formulářů). Po úpravě již nepoužívají statické řetězce, ale odkazují na lokátory z centralizovaného souboru `orsr-selectors.ts`.

Funkce pro **extrakci dat**, jako `getCompanyName`, `getICO`, `getCategories` nebo `getPeopleFromSection`, zodpovídají za parsování HTML struktury konkrétní stránky ORSR a extrakci relevantních informací (název firmy, IČO, adresy, osoby v různých rolích apod.). Tyto funkce se spoléhají na specifickou a neměnnou strukturu stránky. Funkce `extractCompanyInfoFromCurrentPage` následně orchestruje volání těchto dílčích parserů.

Důležitou součástí implementace deterministických funkcí je mechanismus **obohacování chyb**. Standardní výjimky knihovny Playwright (např. `TimeoutError` nebo `ElementHandleError`) poskytují pouze technickou informaci o selhání interakce, ale postrádají sémantický kontext. Pro efektivní diagnostiku pomocí LLM byla proto zavedena strategie **kontextualizovaných výjimek**. Každá deterministická funkce v případě neúspěchu zachytí původní chybu a vyvolá novou výjimku, která obsahuje:

- **Identifikátor funkce:** (např. `[getIC0]`), který lokalizuje místo chyby v logice aplikace.
- **Záměr:** Popis toho, co se funkce pokoušela najít (např. `Section not found`).
- **Data:** Konkrétní hodnotu, která selhala (např. hledaný řetězec selektoru).

Tento přístup transformuje chybový výstup z prostého `element nenalezen` na strukturovaný prompt pro LLM: Funkce `getIC0` selhala, protože na stránce nenašla sekci s nadpisem `'IČO:'`.

Deterministické jádro tak představuje základní metodu pro získávání dat, která je rychlá a efektivní, avšak její spolehlivost je podmíněna neměnností struktury cílové webové stránky. Jak ukázaly experimenty, centralizace lokátorů sama o sobě nestačí, pokud dojde k zásadnější změně struktury nebo logiky stránky. Případné selhání této metody – například při nenalezení očekávaného prvku podle lokátoru nebo při chybě parsování dat – následně iniciuje přechod k záložnímu mechanismu využívajícímu LLM komponentu.

3.3 LLM komponenta

LLM komponenta slouží jako adaptivní vrstva, která se aktivuje při selhání deterministického jádra. Jejím cílem je analyzovat problém a zajistit dokončení úlohy alternativním způsobem. Je implementována především v souboru `llm/orchestrator.ts` a využívá několik klíčových prvků.

Primárním prvkem je **detekce selhání**, která je implementována pomocí `try...catch` bloků ve funkcích `getInfoFromORSRBy*` a

`robustExtractCompanyInfo` v souboru `scraping-orsr.ts`. Zachycené chybové hlášení (např. `timeout`, `nenalezený prvek`, `chyba parsování`) slouží jako klíčový vstup pro LLM.

Na základě typu chyby se aktivuje **dvoustupňový fallback** mechanismus:

1. **Fallback navigace:** Pokud selže deterministická navigace na cílovou stránku, aktivuje se funkce `runNavigationPlanWithFallback`. Tato funkce nejprve sestaví uživatelský prompt obsahující kontext selhání (původní plán, chybové hlášení, aktuální HTML snippet). Tento prompt je následně spolu se systémovým promptem `NAVIGATION_SYSTEM_PROMPT` (definovaným v `prompts.ts`) předán LLM klientovi (`OpenAIStyleClient`). LLM na základě těchto vstupů navrhne opravu původního deterministického plánu (posloupnosti kroků a lokátorů). Cílem je najít novou sekvenci kroků s minimální úpravou, která dosáhne cíle definovaného ve specifikaci (např. `find_listing_by_person.json` nebo `get_detail_by_ico.json`). Výsledkem opravy je log `[LLM-Hint][Navigation][Reason/Fix]`.
2. **Fallback extrakce:** Pokud navigace uspěje, ale selže deterministické parsování dat z cílové stránky (chyba zachycená v `robustExtractCompanyInfo`), aktivuje se funkce `fallbackExtractDetail`. Tato funkce sestaví uživatelský prompt, který obsahuje kompletní HTML kód stránky a původní chybové hlášení z parseru. Tento prompt je následně spolu se systémovým promptem `EXTRACTOR_SYSTEM_PROMPT` předán LLM klientovi. LLM na základě těchto vstupů provede **přímou extrakci dat** z HTML podle schématu definovaného ve specifikaci `extract_company_detail.json`. LLM zde tedy neopravuje kód parseru, ale provádí samotnou extrakci sémanticky. Zároveň analyzuje původní chybu parseru a vrací diagnostickou nápovědu `[LLM-Hint][Extraction][Reason/Fix]`.

Pro komunikaci s externím LLM API slouží **adaptér**, konkrétně třída `OpenAIStyleClient` v souboru `adapter.ts`. Ta zodpovídá za sestavení API požadavku, odeslání, zpracování odpovědi a extrakci JSON dat. Obsahuje i jednoduchou fallback logiku pro případ selhání primárního modelu.

Chování LLM je řízeno pomocí **systému promptů** definovaných v `prompts.ts`. Tyto prompty obsahují instrukce, pravidla, heuristiky pro opravu lokátorů, požadovaný formát výstupu a příklady, jak má LLM reagovat na různé typy chyb. Jejich kvalita je klíčová pro spolehlivost komponenty (23).

Poslední částí jsou **specifikace cílů**, tedy JSON soubory ve složce `llm/specs`. Tyto soubory definují *intent* (záměr), *inputs* (vstupní data), *output* (očekávaný formát výstupu) a *anchors* (preferované/zakázané vzory v lokátorech).

3.4 Orchestrátor

Orchestrátor je zodpovědný za řízení celého procesu a propojení deterministického jádra s LLM komponentou. Klíčovou roli hraje funkce `runWithPostChecks` v souboru `scraping-orsr.ts`.

Primární zodpovědností je **řízení toku**. Jak bylo popsáno, každá hlavní funkce (`getInfoFromORSRBy*`) definuje `deterministicRunner` a `fallbackRunner`. Orchestrátor `runWithPostChecks` nejprve provede pokus o spuštění deterministické cesty. Pokud tato cesta selže, ať už kvůli interní chybě, nebo kvůli neúspěchu následných kontrol, automaticky aktivuje záložní cestu spuštěním `fallbackRunner`.

Po úspěšném získání dat (ať už deterministicky nebo přes LLM) provede orchestrátor **validaci dat** pomocí `validateCompanyBatch`. Tato funkce provádí základní kontroly: ověřuje přítomnost klíčových údajů (IČO, jméno), detekuje duplicity a heuristicky posuzuje relevanci výsledků vůči původnímu dotazu. Při nesplnění kritérií vyhodí chybu `[validation:...]`.

Pokud data projdou validací, následuje **kontrola konzistence** pomocí `checkCompanyHistoryConsistency`. Porovnává identifikátory (IČO, URL, jméno) aktuálně získaných dat s daty z posledního úspěšného běhu (uloženým v souboru `latest.json` v adresáři `history/`). Pokud je poměr překryvu nižší než stanovený limit (např. 30 %), vyhodí chybu `[consistency:...]` jako ochranu proti výrazně odlišným nebo chybným výsledkům. V opačném případě aktualizuje historický soubor.

3.5 Datový tok

Celý proces automatické opravy lze shrnout do následujícího datového toku, řízeného funkcí `runWithPostChecks`:

1. **Spuštění:** Zavolá se jedna z funkcí `getInfoFromORSRBy*`.
2. **Deterministický pokus:** Spustí se `deterministicRunner`, který volá deterministickou navigaci (např. `findDetailByICO`) a extrakci (`robustExtractCompanyInfo` → `extractCompanyInfoFromCurrentPage`).
3. **Detekce selhání extrakce:** Pokud deterministická extrakce selže (krok 2), `robustExtractCompanyInfo` zachytí chybu, spustí LLM extraktor (`fallbackExtractDetail`) a vrátí data. Následuje krok 5.
4. **Detekce selhání navigace/validace:** Pokud selže deterministická navigace (krok 2) nebo následné kontroly (krok 5, 6), hlavní `try...catch` zachytí chybu a spustí se **fallback cesta** (`fallbackRunner`). Ta volá LLM navigátor (`runNavigationPlanWithFallback`). Následně se volá `robustExtractCompanyInfo`, které (pokud deterministický extraktor opět selže) spustí LLM extraktor.
5. **Validace dat:** Získaná data (ať už z kroku 2, 3 nebo 4) jsou validována pomocí `validateCompanyBatch`. Selhání = chyba.
6. **Kontrola konzistence:** Data jsou porovnána s historií pomocí `checkCompanyHistoryConsistency`. Nízký překryv = chyba.
7. **Úspěch:** Data jsou vrácena a uložena jako nový historický snapshot.

3.6 Reprezentace znalostí

Systém explicitně nepoužívá externí bázi znalostí ani fine-tuning modelu. „Znalosti“ jsou reprezentovány implicitně kontextem a explicitně pomocí promptů a specifikací.

Implicitní znalost (kontext) je LLM poskytována v reálném čase. Model získává informace o aktuálním stavu stránky z poskytnutého HTML (HTML_SNIPPET) a o problému z chybové hlášky (FAILED_PARSER_ERROR nebo FAILURE_REPORT).

Explicitní znalost je definována v konfiguračních souborech. Prompty (prompts.ts) obsahují pravidla, heuristiky a příklady, jak má LLM postupovat při opravě plánů nebo extrakci dat (např. preferované typy lokátorů, parsování specifických sekcí). Jedná se o formu „prompt engineeringu“ (23). Specifikace (specs/*.json) definují cíl (*intent*) a především očekávané schéma výstupu. To pomáhá LLM strukturovat odpověď a snižuje riziko halucinací (22).

Způsob reprezentace navržené opravy se liší v závislosti na typu selhání, které LLM komponenta řeší. V případě selhání **navigace** je výsledkem opravy nový JSON objekt typu `Plan`, obsahující upravenou sadu lokátorů a kroků. Rozdíly mezi původním a opraveným plánem jsou zaznamenány pomocí funkce `describePlanDiff`. Oproti tomu při selhání **extrakce** je oprava implicitní: LLM komponenta (`fallbackExtractDetail`) nevrací opravu kódu, ale přímo extrahovaná data ve formátu `CompanyInfo`. Jako doplňková informace je v poli `debug_analysis` vrácen řetězec obsahující diagnostiku původní chyby deterministického parseru a návrh její opravy.

3.7 Omezení návrhu a Shadow DOM

Navržený systém má několik omezení. Jedním z nich je závislost na externím LLM API, která přináší latenci a potenciální nedeterminismus. Klíčovým technickým omezením je však práce se *Shadow DOM*.

Současná implementace LLM extraktoru spoléhá na získání HTML obsahu pomocí `page.content()`, které nevrací vnitřek komponent zapouzdřených v *Shadow DOM*. Podobně LLM navigátor, jenž generuje standardní Playwright lokátory, by selhal, pokud by cílový prvek ležel uvnitř *Shadow DOM*, protože běžné CSS/XPath z kontextu dokumentu jeho hranice nepřekročí. (Sémantické lokátory Playwrightu často zvládnou *open shadow root*, avšak *closed shadow* zůstává nepřístupný.)

Detekce a interakce se *Shadow DOM* vyžadují specifické postupy: rekursivní procházení `shadowRoot`, případně specializovanou syn-

taxi lokátorů v Playwrightu (cílené zanoření) nebo využití exponovaných částí komponent přes `::part`. Současný návrh promptů ani specifikací s touto možností nepočítá. Řešení tohoto problému není součástí této práce, ale představuje přirozené rozšíření: (i) upravit `getRelevantDom` tak, aby rekurzivně serializoval také obsah `Shadow DOM`, (ii) rozšířit systémové prompty a specifikace o pravidla pro práci se `shadow stromy` a generování vhodných lokátorů, a (iii) v praxi preferovat sémantické lokátory nebo domluvené testovací háčky (např. `data-testid`, `::part`) v komponentách, aby byla interakce deterministická i přes zapouzdření.

4 Implementace proof-of-concept

Pro ověření navrženého konceptu z kapitoly 3 byl implementován proof-of-concept (PoC) systém v jazyce TypeScript. Tento systém rozšiřuje existující sadu skriptů pro scraping dat z portálu Obchodného registra Slovenskej republiky (ORSR). Implementace využívá knihovnu Playwright pro automatizaci prohlížeče a rozhraní OpenAI API pro interakci s jazykovým modelem (konkrétně gpt-4o-mini, zvoleným pro jeho rychlost odezvy). Cílem implementace bylo demonstrovat schopnost systému detekovat selhání způsobená změnami lokátorů nebo struktury stránky a automaticky je obejít, případně poskytnout diagnostiku pro jejich opravu.

4.1 Přehled implementace

Implementace sestává z několika klíčových modulů, jejichž struktura byla představena v návrhu. Modul `src/` obsahuje hlavní logiku scraperu, včetně `scraping-orsr.ts` jako jádra scraperu s fallback logikou (`robustExtractCompanyInfo`, `runWithPostChecks`) a hlavními vstupními funkcemi (`getInfoFromORSRBy*`). Dále obsahuje centralizované lokátory (`orsr-selectors.ts`), pomocné utility (`scraping.ts`, `scraping-suite.ts`) a moduly pro následné kontroly (`consistency/`, `validators/`, `types/`).

Modul `llm/` obsahuje komponenty pro interakci s LLM. Klíčový je `orchestrator.ts` (implementace `runNavigationPlanWithFallback` a `fallbackExtractDetail`), `adapter.ts` (třída `OpenAISTyleClient`), `prompts.ts` (definice `NAVIGATION_SYSTEM_PROMPT` a `EXTRACTOR_SYSTEM_PROMPT`) a adresář `specs/` (JSON definice cílů a schémat).

Modul `tests/` obsahuje Playwright testy (`*.spec.ts`) pro ověření funkčnosti. Systém je navržen tak, aby deterministický kód (např. `getCompanyName`) a LLM logika byly co nejvíce odděleny. Funkce `robustExtractCompanyInfo` a `runWithPostChecks` fungují jako adaptéry, které v případě selhání deterministické cesty delegují práci na funkce v `orchestrator.ts`.

4.2 Popis modulů

Implementovaný systém sestává z několika logicky oddělených modulů, které zajišťují jednotlivé části procesu automatické opravy. Následující podkapitoly popisují jejich účel a klíčové komponenty.

4.2.1 `orchestrator.ts`

Tento modul představuje řídicí centrum pro interakci s jazykovým modelem a orchestraci fallbackových scénářů. Jeho hlavní zodpovědností je spouštění navigačních plánů a extrakce dat v situacích, kdy selže primární deterministická metoda. Obsahuje klíčovou funkci `runNavigationPlanWithFallback`, která implementuje obecnou logiku pro spuštění deterministického navigačního plánu, detekci jeho selhání, následné vyžádání opravy od LLM a spuštění opraveného plánu. Pro účely extrakce dat poskytuje funkci `fallbackExtractDetail`, jež na základě poskytnutého HTML obsahu a systémového promptu `EXTRACTOR_SYSTEM_PROMPT` instruuje LLM k provedení sémantické extrakce dat a analýze chyby původního parseru. Modul dále zahrnuje sady pomocných funkcí pro sestavování (`buildDeterministicPlan`), validaci a opravu (`ensureSelectors`, `validateOrRepairSteps`) deterministických plánů specifických pro jednotlivé typy vyhledávání (podle osoby, IČO, názvu firmy). Součástí jsou i utility pro obecnou práci s plány (např. `clonePlan`, `describePlanDiff`) a pro interakci s Playwrightem (`runPlanWithDiagnostics`, `getRelevantDom`). Pro komunikaci s LLM využívá sdílenou instanci klienta `llmClient`.

4.2.2 `adapter.ts`

Modul `adapter.ts` definuje abstraktní rozhraní `LLMClient` pro komunikaci s jazykovým modelem a poskytuje jeho konkrétní implementaci ve třídě `OpenAIStyleClient`. Tato třída zapouzdřuje detaily interakce s OpenAI API (nebo kompatibilním rozhraním), včetně sestavení HTTP požadavku dle specifikace API, odeslání systémového a uživatelského promptu, zpracování odpovědi a ošetření případných chyb. Jako výchozí model byl zvolen `gpt-4o-mini`, především z důvodu jeho rychlosti odezvy, která je klíčová pro plynulý běh fallback mechanismu. Klíčovou metodou je `completeJSON`, která parsuje textovou

odpověď modelu do strukturovaného JSON objektu pomocí funkce `extractJson`. Implementace zahrnuje i jednoduchou fallback logiku pro přepnutí na alternativní model v případě selhání primárního modelu.

4.2.3 `scraping-orsr.ts`

Tento soubor představuje jádro samotného scraperu pro portál ORSR a zároveň integruje deterministickou logiku s fallback mechanismy. Obsahuje hlavní vstupní funkce systému: `getInfoFromORSRByPerson`, `getInfoFromORSRByICO` a `getInfoFromORSRByCompanyName`. Každá z nich definuje specifickou deterministickou (`deterministicRunner`) a záložní (`fallbackRunner`) strategii a využívá centrální funkci `runWithPostChecks` pro jejich řízené spouštění a vyhodnocení. Dále modul implementuje sadu deterministických funkcí pro navigaci (`findListByName`, `findDetailByICO`, `findDetailByCompanyName`), které využívají centralizované selektory z `orsr-selectors.ts` prostřednictvím `page.locator()`. Klíčové jsou rovněž deterministické funkce pro extrakci dat z detailní stránky (`getCompanyName`, `getICO`, `getCategories`, `getPeopleFromSection`, `parseForeignCompany`, `getAddressesByTitleStandalone`, `extractCompanyInfoFromCurrentPage`), které obsahují specifickou logiku parsování HTML a v případě chyby vyhazují výjimku obsahující název funkce pro lepší diagnostiku. Pro zajištění robustnosti extrakce je zde definován wrapper `robustExtractCompanyInfo`, který implementuje `try...catch` blok a v případě selhání deterministické extrakce deleguje práci na LLM extraktor `fallbackExtractDetail`. Funkce `runWithPostChecks` slouží jako hlavní orchestrátor, řídí přepínání mezi deterministickou a fallback cestou a zajišťuje následnou validaci dat a kontrolu konzistence s historií. Modul doplňují pomocné funkce specifické pro ORSR, jako `getInfoFromORSRFromPage` (která využívá `robustExtractCompanyInfo`), `llmListingToCompanyInfos` a `llmDetailToCompanyInfosByIco`.

4.2.4 `prompts.ts`

Definuje a centralizuje systémové prompty (`NAVIGATION_SYSTEM_PROMPT` a `EXTRACTOR_SYSTEM_PROMPT`), které instruují LLM. Také obsahuje funkci `buildPlannerPrompt` pro dynamické sestavení uživatelského

promptu. Detailní analýza obsahu promptů a strategie jejich návrhu je rozebrána v samostatné sekci 4.4.

4.2.5 specs/*.json

Adresář `llm/specs` obsahuje sadu konfiguračních souborů ve formátu JSON, které slouží k formalizaci jednotlivých úloh pro jazykový model. Místo toho, aby byl účel úlohy definován pouze v systémovém promptu, tyto soubory poskytují LLM přesnou, strojově čitelnou definici cíle a očekávaného formátu. Každý soubor specifikace (např. `get_detail_by_ico.json` nebo `extract_company_detail.json`) definuje několik polí:

- `intent`: Stručný textový popis záměru úlohy (např. „Najít výsledkový listing firmy...“).
- `inputs`: Definuje vstupní parametry, které LLM pro splnění úkolu obdrží (např. `ico: "required"`).
- `output`: Definuje očekávaný formát výstupu. Pro LLM extraktor (`fallbackExtractDetail`) je zde definováno přesné JSON schéma (`schema`), které musí vrácená data splňovat (např. pole `companyName`, `ico`, `people` atd.).
- `anchors`: Poskytuje nápovědy pro LLM navigátor. Obsahuje seznamy preferovaných (`prefer`) a zakázaných (`avoid`) textových vzorů nebo typů lokátorů (např. zakazuje `nth-child()`, což pomáhá LLM generovat robustnější plány v souladu s heuristikami definovanými v promptu).
- `success`: Definuje pole podmínek (vyjádřených jako logické výrazy), které lze použít pro následnou validaci výsledků vrácených LLM (např. `results.length >= 1`).

Tyto strukturované specifikace jsou načítány funkcí `loadSpec` a předávány LLM jako součást uživatelského promptu (viz `buildPlannerPrompt`). Jejich rolí je zásadně přispět k omezení halucinací (22) tím, že LLM poskytují jasný rámec a striktní výstupní schéma, jak je blíže popsáno v sekci 4.4.

4.3 Popis pipeline

Celý proces zpracování požadavku, například při volání `getInfoFromORSRByICO`, je řízen funkcí `runWithPostChecks`. Tato funkce nejprve provede pokus o splnění úkolu pomocí deterministické cesty, tedy spuštěním `deterministicRunner`. Pokusí provést jak navigaci (např. `findDetailByICO`), tak následnou extrakci dat voláním wrapperu `robustExtractCompanyInfo`.

Zde může dojít ke dvěma typům selhání. Pokud navigace uspěje, ale selže pouze deterministická **extrakce** dat (např. kvůli změně HTML struktury, kterou parser `extractCompanyInfoFromCurrentPage` nedokáže zpracovat), funkce `robustExtractCompanyInfo` tuto chybu interně zachytí. Transparentně pro zbytek procesu aktivuje LLM **extraktor** (`fallbackExtractDetail`), který získá data sémanticky přímo z HTML. Data získaná touto cestou (deterministická navigace + LLM extrakce) následně pokračují ke kontrolním krokům.

Pokud však selže již samotná deterministická **navigace**, nebo pokud data získaná z předchozího extrakčního fallbacku neprojdou následnými kontrolami, hlavní `try...catch` blok zachytí chybu. Tím se aktivuje kompletní **fallback cesta** spuštěním `fallbackRunner`. Nejprve se spustí LLM **navigátor** (pomocí `runNavigationPlanWithFallback`), který na základě kontextu chyby opraví a provede navigační plán. Po úspěšné LLM navigaci je pro získání dat opět volána funkce `robustExtractCompanyInfo`, která (pokud by deterministický extraktor opět selhal) může znovu interně spustit LLM extraktor.

Data získaná jakoukoliv z těchto cest musí následně projít dvěma společnými kontrolami. Nejprve **validace dat** (`validateCompanyBatch`) ověří základní správnost a relevanci dat vůči původnímu dotazu. Pokud data projdou, následuje **kontrola konzistence** (`checkCompanyHistoryConsistency`), která porovná identifikátory dat s historií v `latest.json`. Pokud je překryv příliš nízký (standardně pod 30 %), je vyvolána chyba `[consistency:...]`, aby se zabránilo uložení potenciálně chybných dat. Teprve pokud data úspěšně projdou oběma těmito kontrolami, jsou považována za platná, vrácena volající funkci a uložena jako nový historický snímek.

4.4 Formát promptů a strategie řízení LLM

Klíčovou komponentou pro řízení chování jazykového modelu v rámci implementovaného systému je soubor `prompts.ts`, který centralizuje definice systémových promptů. Tyto prompty představují sadu detailních instrukcí specifikujících roli modelu, požadovaný formát výstupu, strategii řešení daného úkolu a způsob zpracování vstupních dat. Kvalita a přesnost těchto instrukcí, známá jako *prompt engineering* (23), je sůležitá pro dosažení spolehlivého a předvídatelného chování LLM komponenty. V implementaci jsou definovány dva hlavní systémové prompty:

- **NAVIGATION_SYSTEM_PROMPT:** Tento prompt je určen pro LLM v roli navigátora, konkrétně pro funkci `runNavigationPlanWithFallback`. Jeho úkolem je opravit selhávající deterministický navigační plán. Prompt nejprve definuje roli modelu jako „plánovače pro stabilní scraping“ a striktně vyžaduje výstup ve formátu JSON odpovídající schématu objektu `Plan` (pole `selectors` a `steps`). Důležitou součástí jsou obecná pravidla pro opravu v tzv. „FIX MODE“, která instruují model, aby provedl minimální nutnou úpravu původního plánu (`PREVIOUS_PLAN`) na základě poskytnutého HTML (`HTML_SNIPPET`) a chybového hlášení (`FAILURE_REPORT`). Prompt obsahuje specifické heuristiky pro generování robustních selektorů – preferuje sémantické kotvy jako `role`, `aria-label`, `name` a naopak zakazuje použití křehkých vzorů jako `nth-child` nebo indexované `XPath`. Dále poskytuje doporučení pro jednotlivé typy prvků (navigační odkazy, vstupní pole, tlačítka, řádky výsledků, stránkování), aby LLM volilo co nejstabilnější selektory odolné vůči změnám UI. Součástí promptu je i explicitní seznam povolených názvů selektorů (např. `nav_person_link`, `surname_input`), aby LLM nepoužívalo vlastní názvy. Požadavek na vyplnění pole `assumptions` zajišťuje, že každá úprava plánu je zdůvodněna.
- **EXTRACTOR_SYSTEM_PROMPT:** Tento prompt je navržen pro LLM v roli experta na extrakci dat a ladění parserů, používaného ve funkci `fallbackExtractDetail`. Jeho cílem je přímo extrahovat data z poskytnutého HTML (`HTML_CONTENT`) podle JSON schématu definovaného ve specifikaci

(`extract_company_detail.json`). Prompt zdůrazňuje, že LLM má hledat všechny relevantní sekce a explicitně uvádí příklady očekávaných názvů sekcí na ORSR, včetně jejich variant (např. „Obchodné meno:“ NEBO „Obchodné meno organizačnej zložky:“). Obsahuje specifická pravidla pro parsování polí `people` (včetně správného přiřazení IČO k firmám), `categories`, a také detailní instrukce pro zpracování zvláštního případu sekce „Zahraničná osoba:“. Druhým úkolem tohoto promptu je analyzovat původní chybu deterministického parseru (`FAILED_PARSER_ERROR`). Prompt vyžaduje, aby LLM v poli `debug_analysis` vrátilo dvouřádkový řetězec obsahující popis příčiny selhání (`[LLM-Hint] [Extraction] [Reason]`) a konkrétní návrh opravy kódu (`[LLM-Hint] [Extraction] [Fix]`). Pro zajištění správného formátu a obsahu této analýzy prompt obsahuje několik ****konkrétních příkladů**** (*few-shot learning*) pro různé typy chyb (např. `Section not found, Invalid XPath`) (23).

Modul `prompts.ts` dále obsahuje funkci `buildPlannerPrompt`, která dynamicky sestavuje uživatelskou část promptu pro LLM navigátor. Tím je zajištěno, že model obdrží veškerý potřebný kontext (cíl ze specifikace `*.json`, aktuální stav stránky z `HTML_SNIPPET`, vstupní parametry `GOAL`, případně předchozí plán `PREVIOUS_PLAN` a chybové hlášení `FAILURE_REPORT`) pro úspěšnou opravu navigačního plánu. Použití přesně definovaného JSON schématu v definicích specifikací (`specs/*.json`) je další důležitou součástí strategie řízení LLM, neboť napomáhá zajištění konzistentního a strojově zpracovatelného výstupu, zejména u extrakce dat, a snižuje riziko halucinací (22). Explicitní definice pravidel a omezení v promptech (např. zákaz používání `nth-child`, pravidla pro parsování IČO) rovněž přispívají k omezení nežádoucího chování modelu.

4.5 Integrace s Playwrightem

Integrace s knihovnou Playwright probíhá na několika úrovních, které oddělují přímé provádění příkazů od abstraktnějšího řízení a orchestrace.

Deterministické jádro. Využívá Playwright API pro přímou interakci se stránkou. Funkce jako `findListByName` implementují logiku

pro navigaci pomocí volání `page.goto()` a `page.locator(...).click()`. Parsovací funkce (např. `getCompanyName`) se spoléhají na `page.locator().allInnerTexts()` k extrakci obsahu z prvků nalezených pomocí lokátorů z `orsr-selectors.ts`.

Orchestrátor (Navigace). Funkce `runPlanWithDiagnostics` slouží jako interpret pro JSON objekt typu `Plan`, který je buď deterministicky sestaven, nebo opraven jazykovým modelem. Tato funkce prochází pole `steps` plánu a na základě hodnoty `action` (např. "click", "type") provádí odpovídající Playwright operace, jako `page.locator().click()` nebo `page.locator().fill()`. Tím je oddělena deklarativní definice plánu od jeho imperativního provedení.

Orchestrátor (Extrakce). Na rozdíl od navigace, fallback pro extrakci dat neprovádí jednotlivé Playwright akce.

Funkce `fallbackExtractDetail` využívá Playwright pouze k získání serializovaného stavu stránky pomocí volání `page.content()`. Tento kompletní HTML řetězec je následně (po sanitizaci) předán jako vstupní kontext (`HTML_CONTENT`) jazykovému modelu, který provede sémantickou extrakci dat namísto použití specifických lokátorů.

Testy. Samotné ověřování funkčnosti systému je realizováno pomocí standardních testů v adresáři `tests/`. Tyto testy (např. `test-orsr-get-detail.spec.ts`) využívají vestavěné Playwright *fixtures* (především objekt `page`) a volají hlavní vstupní funkce systému (např. `getInfoFromORSRByICO`). Výsledky vrácené scraperem jsou následně ověřovány pomocí asercí `expect`.

4.6 Postup adaptace existujícího scraperu

Cílem této části je ukázat, jak převést stávající (typicky křehký) scraper na hybridní variantu s deterministickým jádrem a LLM fallbackem. Níže uvedený postup shrnuje manuální kroky, které byly provedeny v rámci této práce.

1. **Centralizace lokátorů.** Vytvoření souboru `orsr-selectors.ts` a přesun všech statických lokátorů (CSS, XPath, role) do jediné

mapy. Tato mapa je záměrně sdílená jak deterministickým kódem, tak LLM větví: deterministické funkce ji používají přímo a LLM navigátor k ní odkazuje skrze *pojmenované klíče* v JSON plánu (`Plan.selectors`). Díky tomu stačí udržovat jeden „zdroj“.

2. **Refaktoring deterministických funkcí.** Navigační a extrakční funkce byly zjemněny a všechny používají `page.locator()` s hodnotami z `orsr-selectors.ts`. Zmenšuje se duplicita a zvyšuje čitelnost.

```
- await page.click('a[href="/orsr/osoba"]')
+ await page.locator(ORSR_SELECTORS.nav_person_link).
  ↪ click()
```

3. **Obohacení chybových hlášek.** Parsery (např. `getCompanyName`) při chybě vyhodí `Error` s prefixem názvu funkce a stručným kontextem (např. hledaný nadpis sekce). To umožňuje LLM přesněji diagnostikovat příčinu.

```
throw new Error("[getICO] Section not found. Tried: '
  ↪ Identifikacni cislo osoby:");
```

4. **Robustní wrapper pro extrakci.** `robustExtractCompanyInfo` obaluje `extractCompanyInfoFromCurrentPage` do `try...catch`; při selhání aktivuje LLM extraktor (`fallbackExtractDetail`), který provede sémantickou extrakci z HTML a přiloží `debug_analysis` (`Reason/Fix`).
5. **Orchestrátor kontrol.** `runWithPostChecks` spouští zvolený běh a následně provede `validateCompanyBatch` a `checkCompanyHistoryConsistency`. Pouze validní a konzistentní výsledek se vrací volajícímu.
6. **LLM komponenty.** Moduly `llm/orchestrator.ts`, `llm/adapter.ts`, `llm/prompts.ts` a `llm/specs/*.json` zajišťují, že LLM navigátor vrací *plán s pojmenovanými lokátory* (`Plan.selectors`), které

odkazují na `orsr-selectors.ts`. Tím je zaručena kompatibilita a auditovatelnost.

```
{
  "selectors": {
    "nav_person_link": "ORSR_SELECTORS.nav_person_link",
    ↪
    "surname_input": "ORSR_SELECTORS.surname_input"
  },
  "steps": [
    { "action": "click", "selector": "nav_person_link"
    ↪ },
    { "action": "type", "selector": "surname_input", "
    ↪ value": "Cuha" }
  ]
}
```

7. **Vstupní body s dvojicí běhů.** Hlavní funkce (`getInfoFromORSRBy*`) byly sjednoceny na dvojici `deterministicRunner/fallbackRunner`, kterou `runWithPostChecks` spravuje a přepíná podle výsledku.

Ukázkový prompt pro navigační opravu (zkrácený). Níže uvedený prompt (`NAVIGATION_SYSTEM_PROMPT`) je *sdílený* mezi `deterministic`-kou a LLM větví díky pojmenovaným lokátorům v `Plan.selectors` (odkazují na `orsr-selectors.ts`). Plná verze je v repozitáři v `llm/prompts.ts`.

```
SYSTEM:
You are a stable scraping planner. Return ONLY valid JSON (
  ↪ no comments, no markdown).
The output MUST conform to this schema:
{
  "selectors": Array<{ "name": string, "type": "role"|"css
  ↪ "text", "value": string }>,
  "steps": Array<{ "action": "type"|"click"|"waitFor"|"delay",
  ↪ "selector"?:string, "text"?:string, "ms"?:number }>,
  "assumptions"?: string
}
```


WARNING: "selectors" is an array (not a map). Allowed
 ↪ actions: type, click, waitFor, delay.

General rules:

- Make minimal necessary changes (FIX MODE). Preserve
 ↪ aliases and step order.
- Primarily fix selector(s) from FAILURE_REPORT; if HTML
 ↪ clearly contradicts others, fix them too.
- Prefer stable anchors: role/aria-label/name/id, unique
 ↪ href; avoid nth-child, brittle XPath.
- Do not combine "role" and "placeholder" in one selector.
- Regex is allowed in "role=...[name=/.../i]", but not
 ↪ inside :has-text().
- In "assumptions", briefly describe EVERY change (what/why).
 ↪

Heuristics:

- nav*_link: prefer role=link[name=/.../i] (or a[href
 ↪ *='...'] as a last resort).
- *_input: for label/name use type:"role"; for placeholder
 ↪ use type:"css".
- search_button: role=button (name containing Hladaj/
 ↪ Vyhledat/Search) or submit input.
- result_rows: rows with a nested <a> to the detail (e.g.,
 ↪ tr:has(a[href*="vypis"| "detail" | "id="])).
- next_page_button: consider only if pagination actually
 ↪ exists.

Allowed selector names (use no others):

- nav_person_link, surname_input, firstname_input
- nav_ico_link, ico_input, aktualny_link, detail_heading
- nav_company_link, company_name_input
- search_button, result_rows, next_page_button

FIX MODE:

- Do not add other keys, do not rename aliases.
- At the failing step, you can add max. 1x waitFor/delay.
- If PREVIOUS_PLAN is provided, do not return an identical
 ↪ one-modify based on HTML_SNIPPET and FAILURE_REPORT.

```

USER:
SPEC=get_detail_by_ico
GOAL={"ico":"53 695 780"}
HTML_SNIPPET=<...relevantni cast DOM...>
PREVIOUS_PLAN={...}
FAILURE_REPORT="Timeout at 'nav_person_link': role=link[name
    ↪ =/jmeno a prijmeni/i]"

# Return only JSON:
{ "selectors": { ... }, "steps": [ ... ], "assumptions":
    ↪ "..." }

```

Pozorování k plně automatické adaptaci. Pokus o „end-to-end“ re-factoring jen přes LLM dal použitelný nástřel (dvojice `deterministicRunner/fallbackRunner`), ale model chyboval v detailech (navázání `try...catch`, názvy funkcí, validace a kontrola konzistence). V praxi se osvědčila kombinace: (i) manuální zjemnění deterministických funkcí a (ii) LLM pro rychlé nalezení robustnějších lokátorů a sémantickou extrakci v chybových případech. Sdílená mapa lokátorů pak zajišťuje, že změna na jednom místě propadne do obou větví.

4.7 Ukázky běhu a diagnostiky

Během vývoje a ladění systému byly pozorovány tři typické scénáře, které demonstrují fungování dvoustupňového fallbacku a schopnost LLM poskytnout diagnostiku i konkrétní návrh změny. Níže jsou uvedeny zkrácené logy z konzole pro každý scénář, doplněné o popis kontextu.

Oprava navigačního lokátoru Tento scénář simuluje situaci, kdy se změní text odkazu na hlavní stránce, což způsobí selhání deterministické navigace. Do souboru `orsr-selectors.ts` byla zavedena záměrná chyba (`/jmeno a prijmeni/i` místo správného `/priezviska a mena osoby/i`).

Výpis 4.1: Log opravy navigačního lokátoru

```
[ORSR] Deterministic path failed: locator.click: Timeout
  ↳ 20000ms exceeded. - waiting for locator('role=link[
  ↳ name=/jmeno a prijmeni/i]')
[ORSR] Using LLM fallback path.
[LLM-Hint][Navigation][Reason] Deterministicky plan (Person)
  ↳ selhal na kroku 0 (click nav_person_link): locator.
  ↳ waitFor: Timeout 10000ms exceeded.
Call log:
  - waiting for locator('role=link[name=/jmeno a prijmeni/i
  ↳ ]').first() to be visible
[LLM-Hint][Navigation][Fix] LLM upravilo plan: selector '
  ↳ nav_person_link':
  role=link[name=/jmeno a prijmeni/i] -> role=link[name=/
  ↳ priezviska a mena osoby/i]
[consistency:fallback] No existing history snapshot, created
  ↳ a new baseline.
```

Deterministická cesta selhala na timeoutu (řádek 1). Orchestrátor aktivoval fallback (řádek 2). LLM navigátor analyzoval chybu (řádek 3–6), porovnal ji s HTML a navrhl konkrétní opravu lokátoru nav_person_link (řádek 7–8). Opravený plán byl úspěšně proveden.

Diagnostika chyby extrakce (neplatný XPath) Tento scénář simuluje syntaktickou chybu v deterministickém parseru. Do funkce getPeopleFromSection byl zaveden neplatný XPath výraz `./tbody/tr/td[`

Výpis 4.2: Log diagnostiky neplatného XPath

```
[Extractor] Deterministic extract failed: [
  ↳ getPeopleFromSection] Invalid XPath in
  ↳ getPeopleFromSection for section "Veduci organizacnej
  ↳ zlozky": locator.allInnerTexts: SyntaxError: Failed
  ↳ to execute 'evaluate' on 'Document': The string './
  ↳ tbody/tr/td[' is not a valid XPath expression.
[Extractor] Using LLM fallback for data extraction.
[LLM-Hint][Extraction][Reason] Parser selhal kvuli
  ↳ neplatnemu/neukoncenemu XPath vyrazu './tbody/tr/td
  ↳ ['.
```

```
[LLM-Hint][Extraction][Fix] V kodu (getPeopleFromSection)
  ↪ dopln chybejici index, aby vznikl platny selektor.
  ↪ Spravny index pro data je [2]. Vysledny XPath bude:
  ↪ `../../tbody/tr/td[2]`.
```

Deterministická extrakce selhala (řádek 1). Díky obohacení chyby v `extractCompanyInfoFromCurrentPage` log přesně uvádí název funkce (`getPeopleFromSection`) i sekci (Vedúci organizačnej zložky:). LLM extraktor byl aktivován (řádek 2) a na základě analýzy chyby navrhl konkrétní opravu XPath (řádek 3–4). Zároveň LLM provedlo sémantickou extrakci dat a test prošel.

Diagnostika chyby extrakce (chybějící sekce) Tento scénář simuluje změnu textu sekce v HTML. V deterministickém parseru `getIC0` byl hledaný název záměrně změněn na neexistující Identifikacni cislo osoby:.

Výpis 4.3: Log diagnostiky chybějící sekce

```
[Extractor] Deterministic extract failed: [getIC0] Section
  ↪ not found. Tried: "Identifikacni cislo osoby:"
[Extractor] Using LLM fallback for data extraction.
[LLM-Hint][Extraction][Reason] Parser selhal, protoze hledal
  ↪ neexistujici sekci "Identifikacni cislo osoby:".
[LLM-Hint][Extraction][Fix] V kodu (getIC0) zmen hledany
  ↪ nazev na "IC0:".
```

Deterministický parser selhal s přesnou chybou (řádek 1). LLM extraktor byl aktivován (řádek 2) a na základě analýzy HTML a chybové hlášky správně identifikoval, že očekávaný text na stránce není, a navrhl správnou opravu – změnit hledaný řetězec na IC0: (řádek 3–4).

Souhrn. Tyto ukázky potvrzují, že implementovaný dvoustupňový LLM fallback je schopen (i) navrhnout minimální úpravu plánu při selhání navigace, (ii) zvládnout přímou sémantickou extrakci v případě chyby deterministického parseru a (iii) vrátit stručnou a praktickou diagnostiku, která vede k malé a cílené opravě deterministického kódu.

5 Experimentální vyhodnocení

Cílem této kapitoly je ověřit funkčnost, spolehlivost a přínosy systému navrženého v kapitole 3 a implementovaného v kapitole 4. Na základě logů z testovacích běhů byla provedena kvantitativní analýza výkonu systému ve třech scénářích selhání v porovnání se základním (baseline) výkonem. Experimenty jsou zaměřeny na schopnost systému automaticky se zotavit ze selhání a poskytnout užitečnou diagnostiku.

5.1 Testovací datová sada

Pro vyhodnocení byly provedeny čtyři sady testů, každá obsahovala 100 běhů (celkem $N=400$). Scénáře byly vybrány tak, aby reprezentovaly nejčastější typy chyb, ke kterým v reálném provozu dochází při drobných změnách frontendových frameworků nebo obsahu:

- **Baseline ($N=100$):** Běh deterministického jádra bez jakékoli chyby pro stanovení základní úrovně výkonu.
- **Scénář Chyba navigace ($N=100$):** Simulovaná chyba v navigační fázi, kdy byl v `orsr-selectors.ts` záměrně zaveden neplatný lokátor pro odkaz na vyhledávání podle osoby.
- **Scénář Chyba sekce ($N=100$):** Simulovaná chyba v deterministickém parseru `getIC0`, který hledal neexistující název sekce.
- **Scénář Chyba XPath ($N=100$):** Simulovaná syntaktická chyba (neukončený výraz) v lokátoru XPath v rámci parseru `getPeopleFromSection`.

5.2 Metodika měření

Pro vyhodnocení výkonu byly sledovány metriky, které jsou klíčové pro nasazení v CI/CD prostředí:

- **Úspěšnost zotavení (Recovery Rate):** Procento scénářů, kdy systém dokázal autonomně dokončit úlohu. Tato metrika je zásadní pro posouzení, zda agent dokáže snížit potřebu okamžitého lidského zásahu (např. noční běhy testů).

- **Kvalita diagnostiky:** Hodnotíme, zda zpráva [LLM-Hint] přesně identifikovala příčinu (např. "změnil se atribut name") oproti vágnímu "element nenalezen". Přesná diagnostika je klíčová pro zrychlení manuální opravy – pokud model pouze "hádá", vývojář musí chybu hledat ručně, čímž se ztrácí přidaná hodnota systému.
- **Časová náročnost (Penalizace):** Srovnání času běhu s fallbackem oproti baseline. V kontextu CI/CD pipeline, kde mohou běžet stovky testů, je extrémní prodloužení času nežádoucí. Měříme, zda je penalizace akceptovatelná výměnou za stabilitu.
- **Analýza selhání:** Klasifikace důvodů neúspěchu. Rozlišujeme mezi technickým selháním (API) a logickým odmítnutím (kontrola konzistence), což demonstruje bezpečnost systému.

5.3 Diskuse výsledků

Experimenty nebyly prováděny manuálně, ale pomocí automatizovaného testovacího skriptu, který v cyklu spouštěl jednotlivé scénáře, simuloval chyby v kódu (injection) a agregoval výsledky do strukturovaných logů. Tato sekce analyzuje naměřené výsledky ze všech 400 běhů.

5.3.1 Souhrnné kvantitativní výsledky

Následující tabulka shrnuje kvantitativní výsledky všech čtyř scénářů. Průměrný čas baseline (4.50s) byl odhadnut z logů úspěšných deterministických běhů.

Tabulka 5.1: Souhrnné výsledky experimentálních běhů (N=400)

Scénář	Počet Běhů	Úspěch (%)	Selhání (%)	Čas (s)	Penal.
1. Baseline	100	100 (100.0%)	0 (0.0%)	4.50 s	-
2. Chyba Navigace	100	98 (98.0%)	2 (2.0%)	54.02 s	~12.0x
3. Chyba Sekce	100	83 (83.0%)	17 (17.0%)	20.30 s	~4.5x
4. Chyba XPath	100	75 (75.0%)	25 (25.0%)	14.82 s	~3.3x

5.3.2 Analýza jednotlivých scénářů

Scénář 1: Baseline (N=100) Deterministický běh bez chyb potvrdil očekávaný výkon. Se 100% úspěšností a průměrným časem ~4.50 sekundy slouží jako spolehlivá základní úroveň pro měření penalizace.

Scénář 2: Chyba navigace (N=100) Tento scénář simuloval kritické selhání, kdy `deterministicRunner` selhal na `timeoutu` při hledání neplatného lokátoru pro navigaci.

Jednalo se o **statickou chybu**, kde byl po celou dobu testu neplatný selektor v kódu. Deterministický běh selhal na **timeoutu** (konkrétně 20 sekund), což je standardní chování Playwrightu. Důležité je, že k tomuto selhání došlo **až při pokusu o interakci s elementem** (kliknutí), nikoli při základním načtení stránky (`page.goto`). Stránka byla úspěšně načtena a LLM tak měl k dispozici kompletní DOM pro analýzu a zotavení.

- **Úspěšnost zotavení:** Systém dosáhl úspěšnosti zotavení **98 %** (98 ze 100 běhů).
- **Kvalita diagnostiky:** Ve všech 98 úspěšných případech LLM navigátor správně identifikoval chybný lokátor a navrhl přesnou sémantickou opravu.
- **Analýza selhání (2 %):** Dva neúspěšné běhy nebyly způsobeny chybou logiky agenta. Jednalo se o externí selhání: 1x chyba API (HTTP 502 Bad Gateway) a 1x timeout při provádění plánu (LLM plan execution timed out).
- **Časová náročnost:** Penalizace za aktivaci LLM navigátoru (`'run-NavigationPlanWithFallback'`) je značná, v průměru **54.02 sekund**, což představuje ~12x nárůst oproti baseline.

Scénář 3: Chyba sekce (N=100) Tento scénář testoval selhání v extrakční logice, kdy se automatický parser pokusil najít sekci s chybným názvem (např. Identifikační číslo osoby: místo IČO:).

- **Úspěšnost zotavení:** Bylo dosaženo **83%** úspěšnosti zotavení (83 ze 100 běhů).

- **Analýza selhání (17 %):** Klíčovým poznatkem je, že vysoký podíl "selhání" představuje **správné zafungování bezpečnostního prvku** (kontroly konzistence). Tato funkce (`checkCompanyHistoryConsistency`) má za úkol porovnat extrahovaná data (výsledek LLM) s historickými daty (seed) a **zabránit přijetí halucinačního řešení**. Jedná se tedy o ten element, který nám zabraňuje akceptovat nefunkční data. V tomto kontextu je "selhání" známkou **správně zavedené auditovatelnosti** a bezpečné architektury.
- **Časová náročnost:** Penalizace byla v průměru **20.30 sekund** (~4.5x nárůst), což je výrazně méně než u chyby navigace.

V aktuální implementaci je selhání kontroly konzistence konečným stavem. Tento design vychází z experimentů s iterativní opravou, při které byl modelu zpětně zasílán zjištěný rozdíl (*diff*) vůči historickým datům. Riziko tohoto přístupu potvrdil test s tzv. **otravou referenčních dat** (*ground truth poisoning*), kdy byla do historie záměrně vložena fiktivní data. Model ve snaze vyřešit předložený diff často zkopíroval tyto neexistující hodnoty do výstupu, jen aby vyhověl požadavku na shodu. Díky detekci pomocí **ukotvení dat** (*data grounding*) byly tyto halucinace odhaleny, což vedlo k upřednostnění bezpečného principu *fail-fast*.

Scénář 4: Chyba XPath (N=100) Tento scénář testoval syntaktickou chybu v parseru `getPeopleFromSection ('.//tbody/tr/td[')`.

- **Úspěšnost zotavení:** Bylo dosaženo **75%** úspěšnosti zotavení (75 ze 100 běhů).
- **Kvalita diagnostiky:** Opět 100% úspěšnost diagnostiky. LLM konzistentně identifikovalo neukončený výraz a navrhlo správnou opravu: [LLM-Hint] [Extraction] [Fix] ... The resulting XPath will be: `'.//tbody/tr/td[2]'`.
- **Analýza selhání (25 %):** Podobně jako u chyby sekce, drtivá většina selhání (**21 z 25, tj. 84 %**) byla způsobena zásahem kontroly konzistence [`consistency: fallback`]. Zbývající 4 selhání byla způsobena chybou `page.goto: Timeout`.

- **Časová náročnost:** Nejnižší penalizace ze sabotovaných scénářů, v průměru **14.82 sekund** (~3.3x nárůst).

5.4 Limity a rizika

Experimenty potvrdily několik limitů návrhu:

- **Latence:** Běh LLM fallbacku (zvláště pokud zahrnuje navigaci i extrakci) je výrazně pomalejší než čistě deterministický běh, jak ukázala analýza časové náročnosti (nárůst 3.3x až 12.0x).
- **Bezpečnost a eliminace halucinací:** Ačkoliv je generování kódu LLM nedeterministické, implementované bezpečnostní mechanismy riziko halucinace efektivně eliminují. Důležitým zjištěním je, že se během experimentů ani v jednom případě nestalo, že by systém akceptoval nesprávnou opravu. Kontrola konzistence (`checkCompanyHistoryConsistency`) systém zastaví (zachytila 36 případů jako nespolehlivé), čímž brání zhoršení stavu aplikace.
- **Potřeba lidské kontroly (Auditovatelnost):** Systém není plně autonomní. I když LLM navrhne funkční opravu, diagnostické výstupy [LLM-Hint] slouží jako **podklady pro lidskou kontrolu a schválení**. Systém tak funguje jako pokročilý asistent, který připraví veškerou nezbytnou dokumentaci a návrh opravy pro rychlé vytvoření **Pull Requestu** na GitHubu. Tím se sice eliminuje repetitivní údržba, ale zachovává se **formální kontrola** a auditovatelnost kódu.
- **Riziko automatických oprav (Sycophancy):** Původní návrh počítal s tím, že se model pokusí chybu sám opravit (architektura *Reflexion*). Experimenty s podvrženými historickými daty však ukázaly, že model pod tlakem na shodu často ignoruje skutečný obsah stránky a raději zkopíruje očekávanou hodnotu z historie, jen aby „vyhověl“ validátoru. Toto chování (tzv. *sycophancy*) by vedlo k zanášení skrytých chyb. Z tohoto důvodu byl proto upřednostněn princip *fail-fast*, systém při neshodě okamžitě zastaví a vyžaduje kontrolu člověkem.

6 Ověření postupu adaptace na systému RPVS

Pro potvrzení obecné platnosti navrženého řešení byla provedena validace na nezávislém externím systému – *Registru partnerov verejného sektora* (RPVS). Tato kapitola slouží jako případová studie, která prakticky demonstuje aplikaci metodiky popsané v sekci 4.6 (Postup adaptace existujícího scraperu). Cílem bylo ověřit, zda lze tento návod použít jako univerzální návod pro převod křehkého scraperu na robustní hybridní systém a to i v prostředí s odlišnými technickými charakteristikami než má ORSR.

6.1 Charakteristika cílového systému

Jako předmět demonstrace byl zvolen portál RPVS (`rpvs.gov.sk`), pro který byla k dispozici existující implementace scraperu. Tento výběr umožnil simulovat reálný scénář údržby a adaptace cizího kódu. Systém navíc vykazuje technické odlišnosti, které prověřily univerzalitu navrženého řešení:

- **Dynamické chování:** Na rozdíl od převážně statického ORSR využívá RPVS intenzivněji JavaScript pro vykreslování výsledků a přechody mezi stránkami.
- **Odlišná DOM struktura:** Identifikátory a rozložení tabulek (např. sekce *Koneční užívatelia výhod*) neodpovídají schématu ORSR.
- **Nestandardní ovládací prvky:** RPVS implementuje interakční prvky způsobem, který neodpovídá běžným konvencím (např. vyhledávací tlačítko nemá atribut `submit`), což představuje výzvu pro obecné heuristiky modelu.

6.2 Aplikace metodiky adaptace

Adaptace byla provedena striktním kroků definovaných v sekci 4.6. Následující body popisují realizaci jednotlivých fází návodu v kontextu RPVS:

6.2.1 Krok 1: Centralizace lokátorů

V souladu s prvním bodem metodiky byl vytvořen nový soubor `rpvs-selectors.ts`. Do něj byly extrahovány všechny původně „hard-coded“ selektory (např. XPath pro tabulky nebo CSS pro vstupní pole).

6.2.2 Krok 2 a 3: Refaktoring a obohacení chyb

Deterministické funkce v `scraping-rpvs.ts` byly přepsány tak, aby využívaly centralizované lokátory. Zásadním krokem pro funkčnost diagnostiky bylo obohacení výjimek. Místo obecných chyb Playwrightu byly implementovány strukturované chyby, např.:

```
throw new Error("[getPeopleFromPanel] Column for 'birth_date  
  ↳ ' not found. Selector: 'xpath=/td[2]' returned no  
  ↳ results.");
```

Tato úprava umožnila LLM v pozdější fázi přesně identifikovat, že selhání nastalo v konkrétní logické části parseru, nikoliv v síťové vrstvě.

6.2.3 Krok 4 a 5: Integrace hybridní vrstvy

Byla implementována funkce `robustExtractPeople`, která funguje jako wrapper pro deterministický parser. Při zachycení chyby automaticky deleguje řízení na existující funkci `fallbackExtractDetail` (z modulu `orchestrator.ts`), které pouze předá URL a HTML obsah RPVS stránky. Pro potřeby navigace byly vytvořeny nové JSON specifikace (např. `extract_rpvs_people.json`), definující očekávané výstupní schéma.

6.3 Zjištění a optimalizace během adaptace

Během aplikace postupu se projevíly specifické problémy cílového systému, které si vyžádaly úpravu konfigurace (nikoliv logiky kódu). Tyto poznatky doplňují obecný návod o praktické aspekty:

Jazyková interference (Halucinace lokalizace). Při generování oprav navigačního plánu model opakovaně navrhoval český výraz „podle“ namísto slovenského „podľa“ v selektoru placeholder. Tento problém

je způsoben vysokou podobností obou jazyků a pravděpodobnou převahou českých textů v trénovacích datech modelu, což vede k tzv. *cross-lingual hallucination*. Řešení: Do systémového promptu pro RPVS byla přidána silná kontextová instrukce vymezující jazyk stránky: „*The page is in Slovak. Do not use Czech keywords (e.g., use 'podľa', NOT 'podle')*“. Tímto zpřesněním kontextu se podařilo modelu vnutit správnou jazykovou sadu pro generování selektorů.

6.4 Výsledky ověření

Po dokončení adaptace a konfigurace byla spuštěna sada 400 validačních testů (100 běhů pro každý ze čtyř scénářů). Tento rozsah odpovídá metodice použité v kapitole 5 pro systém ORSR, což umožňuje přímé porovnání efektivity řešení. Výsledky (viz Tabulka 6.1) potvrzují, že metodika popsaná v sekci 4.6 vede k vytvoření funkčního a odolného systému.

Tabulka 6.1: Výsledky validace adaptovaného RPVS scraperu (N=400)

Scénář	Počet Běhů	Úspěch (%)	Selhání (%)	Čas (s)	Penal.
1. Baseline	100	100 (100.0%)	0 (0.0%)	~6.00 s	-
2. Chyba Navigace	100	98 (98.0%)	2 (2.0%)	~40.00 s	~6.7x
3. Chyba Sekce	100	100 (100.0%)	0 (0.0%)	~13.00 s	~2.2x
4. Chyba XPath	100	100 (100.0%)	0 (0.0%)	~13.00 s	~2.2x

Zejména u scénáře 4 (Syntaktická chyba v XPath) systém prokázal schopnost nejen extrahovat data, ale v diagnostickém výstupu (‘[LLM-Hint]’) přesně navrhnout opravu kódu, což validuje přínos systému pro zrychlení opravy.

6.5 Závěr k adaptaci

Demonstrace navrženého řešení na systému RPVS potvrdila, že postup definovaný v této práci je univerzálně platný. Adaptace nevyžadovala

žádné změny v jádře systému (`orchestrator.ts`, `adapter.ts`), pouze vytvoření nových definic (selektory, specifikace, prompty).

Výsledky validace na systému RPVS navíc vykazují vyšší míru spolehlivosti než u původního experimentu s ORSR. Tento kvalitativní posun lze přičíst především **specializaci promptů**. Zatímco pro ORSR byl použit univerzální extrakční prompt pokrývající celou šíři stránky, pro RPVS byl navržen dedikovaný prompt zaměřený výhradně na sekci *Koneční uživatelů výhod*. Tato dekompozice problému zúžila rozhodovací prostor pro jazykový model a vedla k tomu, že ani v jednom ze 400 testů nedošlo k selhání **kontroly konzistence** (*consistency check*). Bezpečnostní mechanismus, který u ORSR sloužil jako aktivní filtr pro zachytávání halucinací, zde fungoval pouze jako pasivní potvrzení správnosti. To demonstruje, že rozdělení komplexní extrakce na menší, specifické úlohy (agentní přístup) přímo zvyšuje přesnost modelu a eliminuje potřebu opravných iterací.

Tím bylo prokázáno, že navržená architektura *planner-executor-critic* je přenositelná a škálovatelná na různé webové systémy veřejné správy.

7 Budoucí práce

Navržený systém představuje základ, který lze dále rozšiřovat několika směry:

- **Integrace s CI/CD:** Plná integrace do CI/CD pipeline (např. GitHub Actions). Selhaný test by mohl automaticky spustit LLM fallback. Pokud je úspěšný a projde validací, systém by mohl automaticky vytvořit Pull Request s navrženou opravou (např. úpravou v `orsr_selectors.ts`) pro revizi vývojářem.
- **Kombinace se self-healing přístupy:** Rozšíření systému o plně automatické "samo-léčení" (self-healing). Místo pouhého návrhu opravy by systém mohl opravu přímo aplikovat (např. přepsat soubor `orsr_selectors.ts`) a okamžitě spustit testy znovu pro ověření.
- **Adaptivní učení LLM:** Využití historie úspěšných a neúspěšných oprav. Logy `debug_analysis` a aplikované opravy lze sbírat a periodicky je používat k fine-tuningu specializovaného modelu, který by byl v diagnostice a opravách lokátorů ještě přesnější než obecný model.
- **Optimalizace latence a nákladů:** Zkoumání menších, specializovaných nebo lokálně hostovaných modelů (např. Llama 3 8B, Phi-3) pro úlohy diagnostiky. Tím by se mohla výrazně snížit latence (která byla v experimentech až 12x vyšší) a závislost na externích placených API.
- **Podpora pro Shadow DOM:** Rozšíření implementace o podporu pro Shadow DOM, jak bylo popsáno v limitech návrhu, což by systému umožnilo pracovat s moderními webovými komponentami.
- **Dekompozice promptů a specializace agentů:** Případová studie na systému RPVS ukázala, že použití úzce specializovaného promptu (zaměřeného pouze na jednu sekci dat) vede k výrazně vyšší spolehlivosti než použití univerzálního extrakčního promptu, který byl aplikován u ORSR. V budoucnu je proto

vhodné provést refactoring deterministického jádra a rozdělit komplexní extrakční úlohy na menší podúlohy (např. samostatný agent pro adresy, samostatný pro statutární orgány). Tím lze dále snížit riziko halucinací a zvýšit úspěšnost prvního průchodu.

- **Robustní implementace Reflexion architektury:** Současný systém se při neshodě s historií zastaví, protože prosté zaslání rozdílu (*diff*) vede k halucinacím, kdy model opisuje data z historie místo ze stránky (prokázáno testem s *ground truth poisoning*). Budoucí práce by se měla zaměřit na vývoj bezpečnější zpětné vazby, například formou „slepé validace“ (*blind validation*), kdy model dostane informaci o chybě, ale nikoliv očekávanou hodnotu, aby se předešlo úniku informací (*data leakage*) a vynutilo se skutečné opětovné prozkoumání HTML obsahu.

8 Závěr

V této práci jsem navrhnul a implementoval hybridní systém pro automatizované testování a scraping, který kombinuje rychlé deterministické provádění s robustním dvoustupňovým fallback mechanismem založeným na LLM. Implementace proof-of-concept ukázala, že tento přístup dokáže efektivně řešit selhání způsobená drobnými změnami v uživatelském rozhraní.

Hlavní přínosy navrženého řešení jsou:

- **Odolnost (Resilience):** Systém dokáže dokončit úlohu i v případě selhání původních lokátorů nebo parserů.
- **Diagnostika (Diagnostics):** Místo pouhého selhání vrací LLM extraktor konkrétní analýzu příčiny chyby a návrh opravy deterministického kódu, což dramaticky zkracuje čas potřebný k manuální údržbě.
- **Auditovatelnost (Auditability):** Díky logování [LLM-Hint] a kontrole konzistence s historií je zachována kontrola nad automatickými zásahy a je minimalizováno riziko tichého selhání (vracení chybných dat).
- **Ekonomický přínos (Cost-Benefit Trade-off):** Systém efektivně přesouvá náklady z drahé a nepředvídatelné manuální práce vývojáře (analýza logů a oprava) na předvídatelné, i když výpočetně náročnější, náklady v rámci CI/CD (jak ukázala analýza časové náročnosti v kapitole 5).

Bibliografie

1. MICCO, John. *Flaky Tests at Google and How We Mitigate Them* [online]. Google Testing Blog, 2016-05-12. [cit. 2025-10-11]. Dostupné z: <https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html>.
2. PIROCANAC, George. *Test Flakiness – One of the main challenges of automated testing* [online]. Google Testing Blog, 2020-12-10. [cit. 2025-10-11]. Dostupné z: <https://testing.googleblog.com/2020/12/test-flakiness-one-of-main-challenges.html>.
3. *WebDriver* [online]. Selenium Project, 2024-11-07. [cit. 2025-10-14]. Dostupné z: <https://www.selenium.dev/documentation/webdriver/>.
4. *Auto-waiting and actionability* [online]. Microsoft Playwright, 2025. [cit. 2025-10-11]. Dostupné z: <https://playwright.dev/docs/actionability>. Official documentation.
5. *Best Practices* [online]. Cypress.io, 2025. [cit. 2025-10-15]. Dostupné z: <https://docs.cypress.io/guides/references/best-practices>. Recommendations for stable and maintainable tests.
6. BUBECK, Sébastien; CHANDRASEKARAN, Varun; ELDAN, Ronen et al. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. *arXiv preprint arXiv:2303.12712*. 2023. Dostupné také z: <https://arxiv.org/abs/2303.12712>.
7. YAO, Shunyu et al. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint* [online]. 2022 [cit. 2025-10-11]. Dostupné z: <https://arxiv.org/abs/2210.03629>.
8. SHINN, Noah et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint* [online]. 2023 [cit. 2025-10-11]. Dostupné z: <https://arxiv.org/abs/2303.11366>.
9. FOWLER, Martin. *The Test Pyramid* [online]. martinowler.com, 2012. [cit. 2025-10-15]. Dostupné z: <https://martinfowler.com/bliki/TestPyramid.html>. Conceptual model of test level distribution.

10. ZIFTCI, Caglar; MICCO, John, et al. Automatically Locating Root Causes of Flaky Tests in Code at Google. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)* [online]. 2022 [cit. 2025-10-11]. Dostupné z: <https://research.google/pubs/de-flake-your-tests-automatically-locating-root-causes-of-flaky-tests-in-code-at-google/>.
11. *Waiting with Expected Conditions* [online]. Selenium Project, 2025. [cit. 2025-10-11]. Dostupné z: https://www.selenium.dev/documentation/webdriver/support_features/expected_conditions/. Official documentation.
12. *Waits* [online]. Selenium Python Bindings, 2025. [cit. 2025-10-11]. Dostupné z: <https://selenium-python.readthedocs.io/waits.html>.
13. *Locators* [online]. Microsoft Playwright, 2025. [cit. 2025-10-11]. Dostupné z: <https://playwright.dev/docs/locators>. Official documentation.
14. *Trace Viewer* [online]. Microsoft Playwright, 2025. [cit. 2025-10-11]. Dostupné z: <https://playwright.dev/docs/trace-viewer>. Official documentation.
15. *Introduction to Cypress* [online]. Cypress.io, 2025. [cit. 2025-10-15]. Dostupné z: <https://docs.cypress.io/guides/core-concepts/introduction-to-cypress>. Official documentation, Core Concepts.
16. *Intercepting and Mocking Network Requests* [online]. Cypress.io, 2025. [cit. 2025-10-15]. Dostupné z: <https://docs.cypress.io/api/commands/intercept>. Network interception API reference.
17. *Testing Across Multiple Origins* [online]. Cypress.io, 2025. [cit. 2025-10-15]. Dostupné z: <https://docs.cypress.io/guides/end-to-end-testing/multi-origin>. Multi-origin and cross-domain testing guide.
18. *GPT Engineer*. Anton Osika, 2023. Dostupné také z: <https://github.com/AntonOsika/gpt-engineer>. GitHub repository.

19. *SWE-Agent: Agent for Software Engineering Tasks*. Princeton NLP, 2024. Dostupné také z: <https://github.com/princeton-nlp/SWE-agent>. GitHub repository.
20. WANG, Guanzhi et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*. 2023. Dostupné také z: <https://arxiv.org/abs/2305.16291>.
21. *Self-Healing Automation Using AI* [online]. GeeksforGeeks, 2025-07-23. [cit. 2025-10-18]. Dostupné z: <https://www.geeksforgeeks.org/software-testing/self-healing-automation-using-ai/>. Educational article on AI-driven self-healing test automation.
22. *Why Language Models Hallucinate* [online]. OpenAI, 2025-09-05. [cit. 2025-10-18]. Dostupné z: <https://openai.com/index/why-language-models-hallucinate/>. Official research article.
23. *Prompt Engineering Guide* [online]. OpenAI, 2025. [cit. 2025-10-18]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>. Official OpenAI documentation.

A Přílohy

A.1 Šablony systémových promptů

Kompletní znění systémových promptů (NAVIGATION_SYSTEM_PROMPT a EXTRACTOR_SYSTEM_PROMPT) použitých pro řízení LLM v implementaci.

Výpis A.1: Šablony systémových promptů

```
1 // llm/prompts.ts
2 export const NAVIGATION_SYSTEM_PROMPT = [
3   // Role & Output
4   "You are a stable scraping planner. Return ONLY valid JSON
    ↪ (no comments, no markdown, no code blocks).",
5
6   // Target Schema
7   "The output MUST conform to this schema (types and key
    ↪ names):",
8   "{",
9   '  "selectors": Array<{ "name": string, "type": "role" | "
    ↪ css" | "text", "value": string }>,',
10  '  "steps": Array<{ "action": "type" | "click" | "waitFor"
    ↪ | "delay", "selector"?: string, "text"?: string, "ms
    ↪ "?: number }>,',
11  '  "extract"?: Array<{ "name": string, "from": string, "
    ↪ attr": string }>,',
12  '  "assumptions"?: string',
13  "}",
14
15  "WARNING: 'selectors' MUST be an array (not a map object).
    ↪ ",
16  "Only these actions are allowed: type, click, waitFor,
    ↪ delay (case-sensitive).",
17
18  // Working with text
19  "If you need a REGEX for text, use the text engine: value
    ↪ = 'text=/.../i'.",
```

```

20 "DO NOT use Regex inside :has-text(); for :has-text()
    ↳ always use a STRING in quotes (e.g., a:has-text(\"Vyhľad
    ↳ ávanie osoby\")).",
21 "If PREVIOUS_PLAN contains a CSS ':has-text' with a regex,
    ↳ you MUST rewrite it to 'type:\"text\"' or CSS without
    ↳ ':has-text()'.",
22
23 // === General FIX rules for any selector (can be multiple)
    ↳ ===
24 "When a plan fails, fix selectors GENERICALLY and
    ↳ DEDUCTIVELY based on HTML_SNIPPET and FAILURE_REPORT:"
    ↳ ,
25 "- Goal: Minimal change to make the plan run. Preserve
    ↳ alias names (nav_person_link, surname_input,
    ↳ firstname_input, search_button, result_rows,
    ↳ next_page_button).",
26 "- Primarily fix the selector(s) mentioned in
    ↳ FAILURE_REPORT. If HTML_SNIPPET clearly contradicts
    ↳ other selectors (e.g., their text/attribute doesn't
    ↳ exist), fix those too.",
27 "- Prefer STABLE anchors: role/aria-label/label/
    ↳ placeholder/name/id or a unique href.",
28 "- Avoid brittle patterns: do not use nth-child, td:nth-
    ↳ child, xPath, nextSibling, ol/li. For CSS, do not use
    ↳ ':has-text' with regex.",
29 "- You can always change the selector 'type' (role|css|
    ↳ text) if it increases robustness (e.g., from 'text' ->
    ↳ 'css' with href).",
30 "- After fixing, add a brief explanation to 'assumptions'
    ↳ (what changed and why).",
31
32 // === Heuristics for individual aliases (general, not
    ↳ fixed to specific values) ===
33 "For navigation links (nav_person_link, nav_ico_link,
    ↳ nav_company_link):",
34 "- Goal: Find the <a> link for 'Vyhľadávanie osoby', '
    ↳ identifikačného čísla', or 'obchodného mena'.",
35 "- Prefer STABLE selectors: 'role=link[name=/.../i]' or 'a:
    ↳ has-text(\"...\")' with the exact text.",

```

```

36  "- Regex in 'name' (e.g., /identifikačného čísla/i) is
    ↪ allowed, but NOT in :has-text().",
37  "- Use 'href' (e.g., a[href*='osoba']) ONLY AS A LAST
    ↪ RESORT if role/text fails.",
38
39  "For input fields (surname_input, firstname_input, ico_input,
    ↪ company_name_input):",
40  "**RULE 1:** For the 'name' attribute (label), USE 'type:
    ↪ \"role\"'. Always use a regex /.../i.",
41  "  EXAMPLE (Priezvisko): { \"type\": \"role\", \"value\":
    ↪ \"role=textBox[name=/Sem zadajte celé priezvisko/i]\"
    ↪ }",
42  "  EXAMPLE (IČO):          { \"type\": \"role\", \"value\":
    ↪ \"role=textBox[name=/Sem zadajte celé identifikačn/i
    ↪ ]\" }",
43  "  EXAMPLE (Meno firmy): { \"type\": \"role\", \"value\":
    ↪ \"role=textBox[name=/Sem zadajte obchodné meno/i]\" }
    ↪ ",
44  "**RULE 2:** For the 'placeholder' attribute, USE 'type:
    ↪ \"css\"'.",
45  "  EXAMPLE: { \"type\": \"css\", \"value\": \"[placeholder
    ↪ *='Sem zadajte obchodné meno']\" }",
46  "**FORBIDDEN:** NEVER combine 'role' and 'placeholder' in
    ↪ one selector. This is INVALID: 'role=textBox[
    ↪ placeholder=...]'."
47
48  "For 'search_button':",
49  "- Prefer role=button with a name containing 'Hľadať'/'Hl'
    ↪ adať'/'Vyhľadať'/'Search' (language variants).",
50  "- Alternatively: input[type=submit][value*='Hľadať'|...]
    ↪ or CSS for a button/link with corresponding text (no
    ↪ regex in :has-text()).",
51
52  "For 'result_rows':",
53  "- The goal is to select table rows or blocks that contain
    ↪ a LINK to the detail (e.g., href containing 'vypis',
    ↪ 'detail', 'firma', 'id').",
54  "- Use something like: tr:has(a[href*='vypis']), tr:has(a[
    ↪ href*='detail']), or more generally tr:has(a[href*='id
    ↪ ='])).",

```

```

55  "- If the page is not table-based, use a robust block (e.g
    ↪ ., article, li) with a nested <a> to the detail.",
56
57  "For 'next_page_button':",
58  "- Allow pagination only if demonstrably present. Look for
    ↪ links with text 'Ďalšia strana', '>>', page numbers,
    ↪ or common 'href' patterns (e.g., 'page=' parameter,
    ↪ etc.).",
59  "- In the execution step, the click on 'next_page_button'
    ↪ is conditional (no-op if the element is not present)."
    ↪ ,

60
61  // Allowed selector names
62  "Use EXCLUSIVELY these selector NAMES (do not use others):
    ↪ ",
63  // Search by person
64  "- nav_person_link",
65  "- surname_input",
66  "- firstname_input (optional)",
67  // Search by ICO
68  "- nav_ico_link",
69  "- ico_input",
70  "- aktualny_link", // for clicking the 'Aktuálny' link
71  "- detail_heading", // to verify detail page load
72  // Search by company name
73  "- nav_company_link",
74  "- company_name_input",
75  // Common
76  "- search_button",
77  "- result_rows",
78  "- next_page_button (pagination)",
79
80  // Robustness & FIX MODE
81  "Do not add other keys. Do not rename selectors. Do not
    ↪ use aliases like lastNameInput etc.",
82  "FIX MODE: Fix the minimal NECESSARY number of selectors
    ↪ to make the plan work. Always preserve alias names and
    ↪ step order. You can add max. 1 'waitFor'/'delay' at
    ↪ the failing step.",

```

```
83 "If PREVIOUS_PLAN is provided, you must not return an
    ↳ identical plan - modify relevant selectors/steps to
    ↳ match the current HTML (read HTML_SNIPPET and use
    ↳ current text/attributes).",
84 "In the 'assumptions' field, briefly describe EVERY change
    ↳ (what changed and why).",
85 ].join("\n");
86
87 export const EXTRACTOR_SYSTEM_PROMPT = [
88 "You are an expert data extractor and parser debugger.",
89 "Your task is to extract data EXACTLY according to the
    ↳ JSON schema 'spec.output.schema.data'.",
90
91 "Analyze the entire HTML and find ALL relevant sections
    ↳ for 'people' and other data.",
92 "WARNING: Section names may vary! Look for:",
93 " - 'Obchodné meno:' OR 'Obchodné meno organizačnej zlož
    ↳ ky:' (for companyName)",
94 " - 'Sídlo:' OR 'Sídlo organizačnej zložky:' (for
    ↳ addresses)",
95 " - 'IČO:' (for ico)",
96 " - 'Predmet podnikania (činnosti):' (for categories)",
97 " - 'Spoločníci:', 'Vedúci organizačnej zložky:', 'Štatut
    ↳ áry orgán:', 'Dozorná rada:', 'Prokúra:', '
    ↳ Komanditisti:', 'Komplementári:' (these contain
    ↳ PERSONS or COMPANIES and belong in the 'people' array)
    ↳ ",
98 " - 'Zahraničná osoba:' (this section can contain a
    ↳ COMPANY or PERSONS, both belong in 'people')",
99
100 "**RULES FOR PARSING 'people' (apply to ALL relevant
    ↳ sections):**",
101 " - From each person/company, extract 'name' and 'role'.",
    ↳
102 " - If an address is listed for the person/company (
    ↳ usually on lines below the name), extract 'street', '
    ↳ city', 'country'.",
103 " - **WARNING: ICO IN PERSON SECTIONS:**",
```



```
104 "    - If a line contains 'Jméno Firmy, s. r. o., IČO:
    ↪ 12345678', extract the ICO into 'ico' for THIS COMPANY
    ↪ .",
105 "    - If the ICO is on a separate line BELOW the company
    ↪ name, assign it to THIS COMPANY.",
106 "    - DO NOT assign a company's ICO to a PERSON listed
    ↪ after it.",
107
108 "**RULES FOR PARSING 'categories' (from section 'Predmet
    ↪ podnikania (činnosti):'):**",
109 "    - The text in this section may contain multiple lines.",
    ↪
110 "    - Each line represents one category.",
111 "    - Ignore lines that only contain a date in parentheses
    ↪ (e.g., '(od: 24.04.2021)').",
112 "    - The result MUST be an array of strings (Array<string>)
    ↪ .",
113
114 "**SPECIAL RULES FOR SECTION 'Zahraničná osoba:**",
115 "    - If it contains a COMPANY ('Obchodné meno:', 'Sídlo:',
    ↪ 'Právna forma:...'), extract it as ONE entry in '
    ↪ people':",
116 "        - Put 'Obchodné meno:' into 'name'.",
117 "        - Split 'Sídlo:' into 'street', 'city', 'country'.",
118 "        - Set 'role' to 'zahraničná osoba'.",
119 "    - If the section also contains PERSONS (e.g., its
    ↪ representatives), extract them as ADDITIONAL entries
    ↪ in 'people' with the role 'zahraničná osoba' and their
    ↪ address.",
120
121 "If the user provides a 'FAILED_PARSER_ERROR', you MUST
    ↪ also fill the 'debug_analysis' field.",
122 "The 'debug_analysis' field MUST be ONE SINGLE STRING.
    ↪ NEVER return an object.",
123 "In this string, describe TWO points:",
124 "    1. WHY the parser failed (identify the PRECISE cause).",
    ↪
125 "    2. A proposal for a SPECIFIC FIX (what exactly to
    ↪ change in the code).",
126
```

```

127 "The 'debug_analysis' string MUST contain TWO lines
    ↳ separated by a newline character ('\n'):",
128 "  1. line: '[LLM-Hint][Extraction][Reason] <WHY parser
    ↳ failed>'",
129 "  2. line: '[LLM-Hint][Extraction][Fix] <PROPOSAL FOR
    ↳ SPECIFIC FIX>'",
130
131 "**EXAMPLES FOR 'debug_analysis':**",
132 "  - Example 1 (Section not found):",
133 "    '[LLM-Hint][Extraction][Reason] Parser failed because
    ↳ it searched for a non-existent section \"Obchodné
    ↳ meno-Sabotaz:\".\n[LLM-Hint][Extraction][Fix] In the
    ↳ code (e.g., getCompanyName), change the searched name
    ↳ to \"Obchodné meno:\".'",
134 "  - Example 2 (Invalid XPath):",
135 "    If FAILED_PARSER_ERROR='Invalid XPath in
    ↳ getPeopleFromSection... The string \"'./tbody/tr/td[\"
    ↳ is not a valid XPath expression.':",
136 "    => '[LLM-Hint][Extraction][Reason] Parser failed due
    ↳ to an invalid/unterminated XPath expression \"'./tbody
    ↳ /tr/td[\".\n[LLM-Hint][Extraction][Fix] In the code (
    ↳ getPeopleFromSection), add the missing index to create
    ↳ a valid selector. The correct index for data is [2].
    ↳ The resulting XPath will be: \"'./tbody/tr/td[2]`. '",
137 "  - Example 3 (Could not extract text):",
138 "    '[LLM-Hint][Extraction][Reason] Parser failed because
    ↳ XPath \"'./tbody/tr/td[2]` found no text (probably
    ↳ returned an empty array).\n[LLM-Hint][Extraction][Fix
    ↳ ] Check the XPath and HTML structure of the section; a
    ↳ different selector or index might be needed.'",
139
140 "Return ONLY the resulting JSON object.",
141 ].join("\n");

```

].

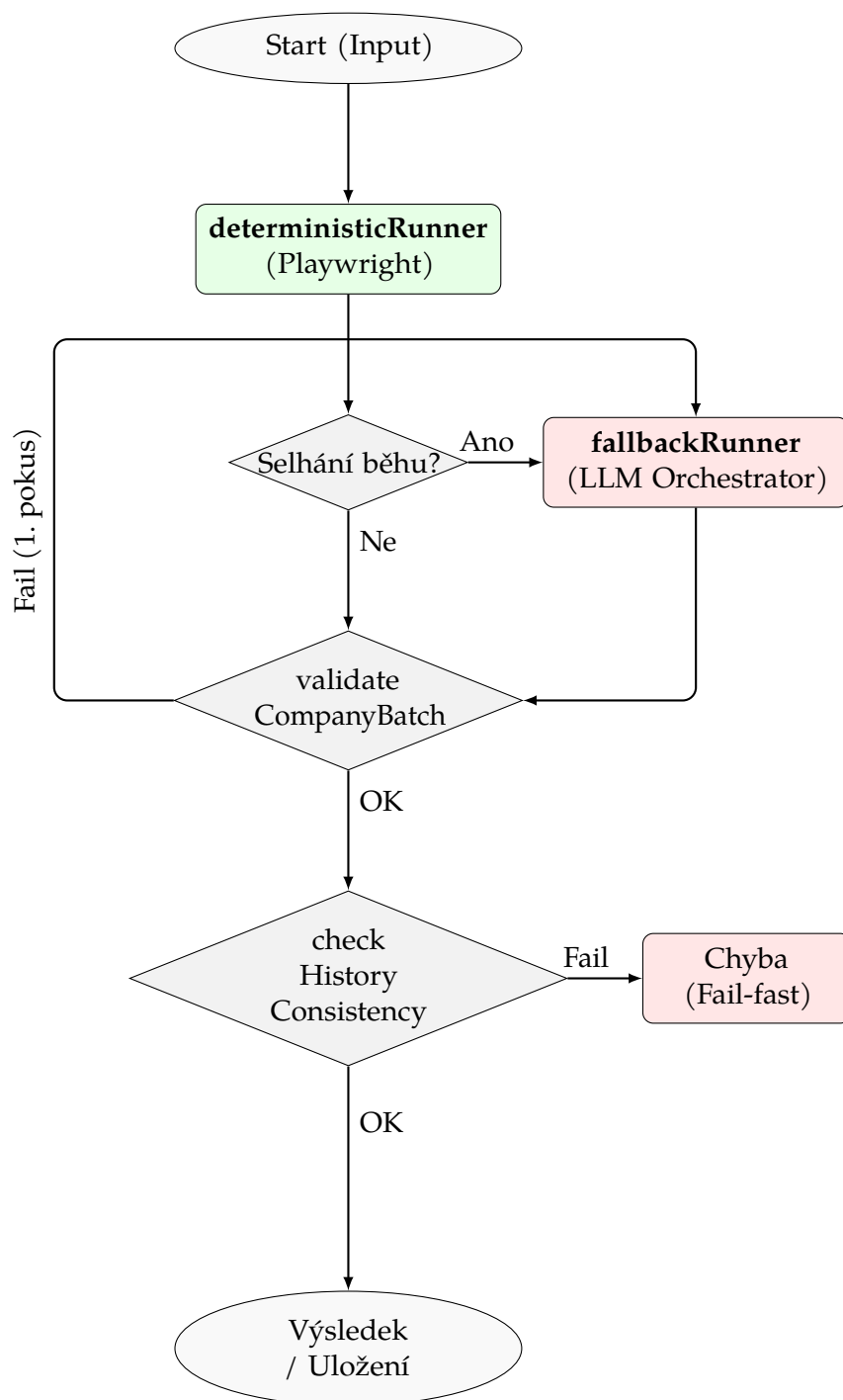
A.2 Časová náročnost

Tabulka A.1: Srovnání časové náročnosti opravy chyby (ORSR)

Metoda	Popis procesu	Čas (cca)
Deterministický běh	Standardní průchod bez chyby (Baseline).	4,5 s
LLM Fallback (Navigace)	Detekce chyby → Generování plánu → Exekuce (Scénář 2).	54,0 s
LLM Fallback (Extrakce)	Detekce chyby → Analýza HTML → Extrakce JSON (průměr Scénářů 3 a 4).	17,6 s

A.3 Schéma architektury systému

Diagram znázorňuje datový tok v implementovaném systému. Zelená cesta představuje rychlý deterministický průchod, červená cesta aktivaci LLM agenta.



Obrázek A.1: Schéma datového toku a kontrolních mechanismů v systému

A.4 Obsah elektronické přílohy

Součástí této diplomové práce je elektronická příloha v archivu IS MU, která obsahuje kompletní zdrojové kódy implementovaného řešení. Adresářová struktura přílohy je následující:

- `src/` – Zdrojové kódy scraperu a deterministického jádra (TypeScript).
- `llm/` – Implementace orchestrátoru a definice promptů.
- `tests/` – Sada integračních testů (Playwright) použita pro experimenty.
- `history/` – Vzorová historická data pro kontrolu konzistence.
- `README.md` – Návod na instalaci a spuštění projektu.